

PicoVISTA: Small Policy, Selective Reasoning for Robust On-Device Robot Manipulation

ABSTRACT

Foundation vision-language-action models (VLAs) offer general-purpose robotic control but incur prohibitive latency, memory, and energy costs on edge devices. Yet fixed deployments typically involve narrow, repetitive workloads, creating a *generality-deployment mismatch*. We introduce PicoVISTA, an ultra-lightweight on-device framework that combines workload specialization with selective reasoning for risky decisions. PicoVISTA distills a foundation VLA into a compact stochastic Student through representation and action distillation, on-policy correction, and world-model-based consequence alignment. At runtime, additional action candidates are evaluated only when the nominal action is deemed risky. A dual-view latent world model predicts candidate outcomes, task-semantic and continuation-value models estimate short- and long-term success, and a rescue-harm-aware gate selects an alternative only when it is expected to improve the outcome without disrupting likely success. The complete **78M**-parameter system runs entirely on a Rubik Pi. Under matched FP32 profiling on an NVIDIA H20, PicoVISTA reduces peak GPU memory, decision latency, and energy per request by 69.7 $\times$, 10.2 $\times$, and 20.7 $\times$, respectively, relative to its $\pi_{0.5}$ Teacher. It achieves 96.4% average success on LIBERO, within 0.5 percentage points of the Teacher, and 89.8% across eight real-world single- and bimanual tasks. Selective intervention further improves LIBERO success by 6.3 points over the Student alone while reducing cumulative neural inference time by 76.7% compared with evaluating alternatives at every decision.

The project page is available at <https://picovista.github.io/>.

1 INTRODUCTION

Embodied intelligence is shifting from the cloud to autonomous robots, where latency, energy, memory, privacy, and reliability constraints demand on-device execution. Foundation vision-language-action models (VLAs) enable general-purpose control across diverse scenes, instructions, tasks, objects, and embodiments [1, 2, 3, 4]. However, this generality incurs substantial overhead, and fewer parameters do not necessarily ensure faster or more energy-efficient inference.

This overhead, which we term the *generality tax*, is particularly excessive in fixed deployments, where robots typically operate in stable environments with limited objects and repetitive tasks. A preliminary analysis of 20,080 observations from CALVIN and LIBERO (embodied AI datasets) shows that these workloads occupy localized regions of the VLA representation space, with distances increasing across tasks, suites, and simulators. This reveals a fundamental *generality-deployment mismatch*: fixed workloads repeatedly

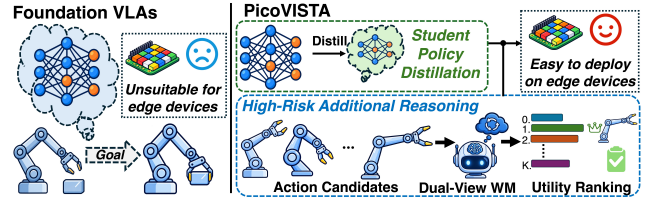


Figure 1: Left: Large VLAs are difficult to deploy on edge devices. Right: PicoVISTA uses a compact Student for routine control and invokes additional reasoning only for high-risk decisions.

revisit localized regions of a foundation VLA’s representation space while incurring its full inference cost

These observations reveal a fundamental *generality & deployment mismatch*: foundation VLAs incur the full cost of generality at every control step, whereas fixed deployments operate within narrow task distributions. This locality creates an opportunity for specialization but does not eliminate the need for foundation knowledge: lightweight policies trained from scratch remain substantially less effective than those using frozen foundation representations. Effective specialization must therefore distill deployment-relevant knowledge rather than relearn the target policy from scratch. Yet aggressive distillation can reduce robustness to ambiguous observations, contact-sensitive transitions, execution errors, and rare states. Moreover, standard imitation distillation overlooks action consequences, while planning at every step offsets the efficiency gains of specialization. The challenge is to retain foundation knowledge in a compact policy and invoke additional reasoning only when failure is likely.

To address this challenge, we propose PicoVISTA, an ultra-lightweight embodied intelligence framework for autonomous on-device robot control. PicoVISTA follows a *distill for routine control, reason on demand* paradigm: a compact stochastic policy handles nominal decisions, while additional action candidates are generated and evaluated only when the nominal action is predicted to be unreliable. By integrating dual-view future prediction, short-term semantic progress estimation, long-term continuation value, and rescue-harm-aware intervention, PicoVISTA selectively corrects risky decisions without incurring foundation-model inference or expensive planning at every control step.

To build the compact policy, PicoVISTA adopts a three-stage distillation scheme. Representation and action distillation transfer the Teacher’s visual-language representations and control behavior, respectively. On-policy refinement then queries the Teacher at states visited by the Student, mitigating the distribution shift between offline training and closed-loop deployment. During this stage, a frozen world

model aligns the predicted outcomes of Student and Teacher actions, enabling the Student to preserve both immediate control behavior and its future consequences.

To assess actions before execution, a dual-view latent world model predicts short-horizon outcomes from complementary agent and wrist views. The agent view captures global context, while the wrist view provides local manipulation and contact information. Two models evaluate predicted trajectories: a task-semantic model measures short-term progress using task-grounded predicates, and an action-conditioned continuation-value model estimates eventual success after the action prefix and return to Student control. Together, they distinguish immediate progress from long-term completion.

At each decision step, PicoVISTA first evaluates the nominal Student action. A high-value proposal is executed directly, while a low-value proposal triggers the generation and evaluation of alternatives. A rescue-harm-aware gate then decides whether to intervene by jointly estimating an alternative’s potential to rescue a nominal failure, harm a nominal success, and improve the final outcome. The nominal action is replaced only when an alternative is predicted to be both beneficial and safe; otherwise, the gate abstains. This selective design concentrates additional computation and intervention on decisions where they are most needed.

By combining compact control, risk-triggered reasoning, and benefit-aware intervention, PicoVISTA improves efficiency at the model, decision, and execution levels. The complete 78M-parameter system runs entirely on a Rubik Pi via Qualcomm AI Engine Direct, supporting candidate generation, future prediction, action evaluation, and receding-horizon control without cloud inference. Compared with its $\pi_{0.5}$ Teacher, PicoVISTA uses 46.4 $\times$ fewer inference-time parameters and, under matched FP32 profiling on an NVIDIA H20, reduces peak memory, decision latency, and energy per request by 69.7 $\times$, 10.2 $\times$, and 20.7 $\times$, respectively. It achieves 96.4% average success on LIBERO, within 0.5 points of the Teacher, and 89.8% across eight real-world single- and bi-manual tasks. Compared with the compact Student alone, selective intervention improves LIBERO success by 6.3 points and matches always-on evaluation while reducing cumulative neural inference time by 76.7%.

Our contributions are summarized as follows:

- We propose PicoVISTA, an ultra-lightweight on-device embodied intelligence framework that *distills for routine control and reasons on demand*, focusing additional computation on high-risk decisions.
- We develop future-aware distillation and action evaluation through on-policy correction, world-model alignment, dual-view prediction, semantic progress, and action-conditioned continuation values.
- We design a rescue-harm-aware gate that intervenes only when an alternative is predicted to rescue failure without disrupting nominal success.

- We deploy the complete 78M-parameter PicoVISTA on a Rubik Pi. It achieves 96.4% average success on LIBERO—within 0.5 percentage points of its $\pi_{0.5}$ Teacher—and 89.8% across eight real-world tasks.

2 RELATED WORK

2.1 Vision–Language–Action Models

2.1.1 Normal-scale VLA. VLA models generate discrete actions or continuous action chunks from images, language, and proprioception. Here, we focus on medium- and small-scale models. RT-2 adapts PaLM-E-12B [5] or PaLI-X-55B [6] for autoregressive action-token prediction, transferring Internet scale knowledge at high memory and latency costs [7]. OpenVLA (~7B) combines DINOv2–SigLIP encoders, a Llama-2 backbone, and quantized action tokens, offering an open alternative that remains costly for real-time edge deployment [1]. Recent models increasingly replace discrete action tokens with continuous generative action heads. RDT-1B (~1.2B) uses a multimodal diffusion Transformer for bimanual action generation[8]. π_0 (~3.3B) combines a PaliGemma-derived backbone with a ~300M flow-matching action expert, while $\pi_{0.5}$ uses a similar architecture for improved open-world generalization[9, 2]. GR00T N1 (~2B) combines VLM reasoning with diffusion-based humanoid control[10]. Helix likewise separates low-frequency reasoning from high-frequency visuomotor control, while Gemini Robotics adds physical actions to a Gemini backbone; neither discloses full parameter counts [11, 12].

2.1.2 Lightweight and Edge-deployable VLA. To deploy VLA models on robots with limited memory, power, and computational resources. Several lightweight VLA and generalist policy models have been developed. RT-1 (~35M) combines EfficientNet-B3, FiLM, TokenLearner, and a Transformer decoder, achieving efficient multi-task control but limited open-world generalization [13]. FAST (~ 2.92B) [14] focuses on efficient action tokenization for VLA policies. 3D-VLA (~ 4.1B) and Hi Robot (~ 3.3B) extend VLA-style control toward 3D grounding, mobile manipulation, and hierarchical instruction following [15, 16]. Octo-Small/Base contain approximately 27M/93M parameters and use compact Transformers with diffusion action heads [17]. VIMA (up to ~200M) reduces visual complexity through object-centric multimodal prompts[18], while SmoVLA (~450M) employs a compact VLM and action expert [19]. However, lightweight VLA methods reduce memory and latency but may compromise generalization, spatial precision, responsiveness, and robustness [20, 1, 21, 22].

2.2 Runtime Verification and Correction

Existing studies improve VLA task success through failure detection, recovery, and proactive verification. Given limited real-world failure data, most detect execution deviations from deployment observations using conformal prediction [23, 24], state-based anomaly detection [25, 26], or

LLM/VLM-based failure recognition [27, 28, 29]. These approaches offer confidence guarantees, efficient monitoring, and semantic failure recognition, respectively, but mainly detect failures after they occur and lack a closed loop for autonomous recovery. Post-hoc analysis and residual policies can correct some failures [30, 31], but recovery is difficult after object drops, collisions, or major environmental changes.

Proactive runtime verification enables earlier intervention. Visual world models (WMs) [32, 33, 34] predict action-induced future states to anticipate risks. EV-WM [35] converts predicted states into task events and selects actions based on progress, semantic consistency, physical plausibility, and uncertainty. VLM-based methods instead predict failures from observations, instructions, and task progress. CycleVLA [36] detects failures at subtask transitions and self-corrects through backtracking, retries, and MBR decoding. Unlike human-assisted PAINT [25] and value-guided retrievals [37], these methods improve autonomous self-correction. However, they remain limited by hallucinations, latency, inaccurate backtracking, retry costs, irreversible failures, and uncertain real-world generalization.

3 PRELIMINARY STUDY

We first characterize the system-level tension that motivates deployment-specific VLA inference. Foundation VLAs retain broad cross-task capacity at substantial online cost, whereas fixed deployments repeatedly exercise narrower regions of that capacity. We study three questions: how large this *generality tax* is, whether foundation representations exhibit deployment locality, and whether target-workload training alone can recover the relevant foundation knowledge.

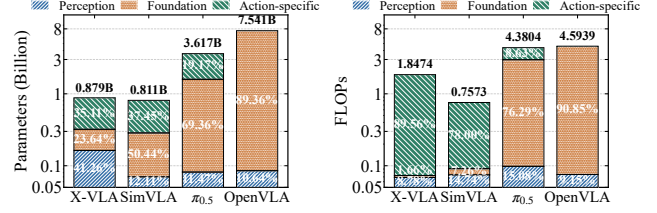
3.1 Generality Tax in Foundation VLAs

Foundation VLAs are designed to generalize across scenes, instructions, tasks, objects, and robot embodiments [1, 2, 3, 4]. This generality requires substantial perception, reasoning, and action-generation capacity to remain in the inference path. We refer to the resulting model footprint and online cost as the *generality tax*.

We profile four representative models with distinct action-generation mechanisms: SimVLA (0.811B parameters), X-VLA (0.879B), $\pi_{0.5}$ (3.617B), and OpenVLA (7.541B). All models are evaluated on a single NVIDIA H20 GPU under consistent inference settings, using batch size one and a single RGB observation per request.

We group model components into *perception*, *foundation*, and *action-specific* modules. As shown in Fig. 2(a), action-specific modules account for 37.45% and 35.11% of SimVLA and X-VLA parameters, respectively, whereas shared backbones account for 69.36% of $\pi_{0.5}$ and 89.36% of OpenVLA. OpenVLA generates actions through its shared language-model path and therefore has no separate action-specific block in this decomposition.

Per-request computation ranges from 0.7573 to 4.5939 TFLOPs (Fig. 2(b)). Action generation accounts for 78.00%



(a) Parameter composition (b) FLOPs composition
Figure 2: Cost decomposition of representative VLAs. Parameters and per-request FLOPs are divided into perception, foundation, and action-specific components.

Table 1: Runtime cost under FP32 inference.

Model	E2E (ms)	Req. rate (Hz)	Peak Mem. (GiB)	GPU Energy (J/req.)
SimVLA	101.35	9.87	3.60	18.45
X-VLA	180.99	5.53	4.00	38.13
$\pi_{0.5}$	317.01	3.15	18.11	63.40
OpenVLA	321.63	3.11	29.55	99.05
PicoVISTA	31.21	32.04	0.26	3.06

and 89.56% of the FLOPs in SimVLA and X-VLA, respectively, whereas the shared backbone dominates $\pi_{0.5}$ and OpenVLA, contributing 76.29% and 90.85%. Notably, computational cost does not scale directly with model size: X-VLA requires 2.44× the FLOPs of SimVLA despite their similar parameter counts, while OpenVLA and $\pi_{0.5}$ incur comparable FLOPs despite a more than 2× difference in model size.

As shown in Tab. 1, E2E latency ranges from 101 to 322 ms per request, corresponding to 3.11–9.87 Hz even on an H20. Peak GPU memory ranges from 3.60 to 29.55 GiB, while energy consumption reaches 18.45–99.05 J per request. These results reveal substantial and highly model-dependent runtime overheads beyond computation alone.

Observation 1: Foundation VLAs exhibit substantial and architecture-dependent generality taxes: each request consumes 0.76–4.59 TFLOPs, 3.60–29.55 GiB of GPU memory, 101–322 ms, and 18–99 J.

Insight 1: These substantial online costs motivate deployment-specific specialization if fixed deployments occupy only localized regions of a foundation VLA’s representation space.

3.2 Deployment Locality in VLA Representations

We next examine whether robotic workloads remain localized in a foundation VLA’s representation space under increasing task and deployment shifts.

We collect 20,080 observations from 34 CALVIN tasks [38] and 40 tasks across the four LIBERO suites [39]. Each observation is processed by the same frozen foundation VLA to extract its fused task representation $\mathbf{r}_i^{\text{VLA}}$. We measure representation similarity using cosine distance:

$$d_{\cos}(i, j) = 1 - \frac{(\mathbf{r}_i^{\text{VLA}})^{\top} \mathbf{r}_j^{\text{VLA}}}{\|\mathbf{r}_i^{\text{VLA}}\|_2 \|\mathbf{r}_j^{\text{VLA}}\|_2}.$$

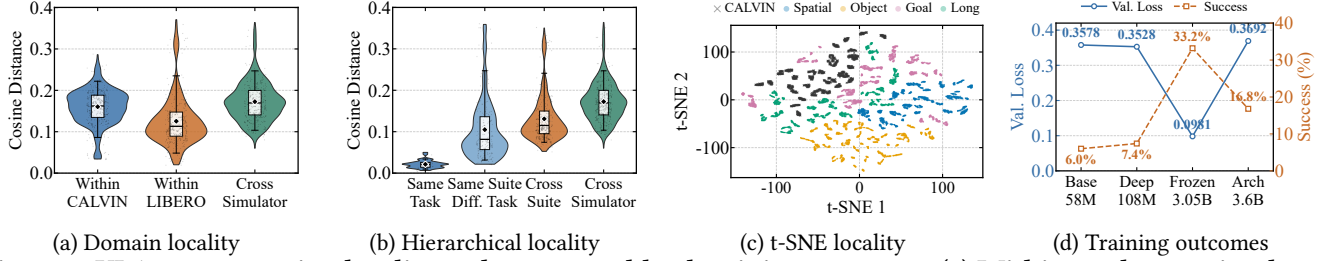


Figure 3: VLA representation locality and target-workload training outcomes. (a) Within- and cross-simulator cosine distances. (b) Hierarchical distances across tasks, suites, and simulators; each distribution contains exact per-stratum means, and diamonds mark category means. (c) t-SNE projection of all 20,080 representations. (d) Validation loss and closed-loop task success for models with different parameter scales and representation sources.

We compare four levels of locality: *same task*, *same suite*, *cross suite*, and *cross simulator*. Same-task pairs are sampled across different episodes to exclude temporal continuity.

Fig. 3(a) shows that representation distance increases across deployment domains: the mean distance is 0.125 within LIBERO, 0.160 within CALVIN, and 0.173 across simulators. Thus, even under the same frozen checkpoint, observations from the same deployment occupy more localized representation regions.

The finer-grained comparison in Fig. 3(b) reveals a consistent hierarchy: $D_{\text{same task}} < D_{\text{same suite}} < D_{\text{cross suite}} < D_{\text{cross simulator}}$. Relative to the same-task distance, the latter three are 4.92 $\times$, 6.15 $\times$, and 8.10 $\times$ larger, respectively. Repeated executions of the same task therefore occupy compact regions, with representation shifts growing progressively across task families and deployment domains.

The t-SNE projection in Fig. 3(c) corroborates this structure: CALVIN and the four LIBERO suites form distinct but partially overlapping neighborhoods, with additional suite-level organization within LIBERO.

Observation 2: Fixed robotic workloads show strong representation locality: distances increase from same-task to cross-simulator comparisons, exceeding 8 $\times$ the same-task distance across simulators.

Insight 2: Together with Sec. 3.1, this reveals a *generality-deployment mismatch*: foundation VLAs repeatedly execute their full general-purpose inference path despite operating within localized representation regions. This locality creates an opportunity for deployment-specific specialization; we next examine whether a compact policy can capture the required knowledge from deployment data.

3.3 Limits of Target-Workload Training from Scratch

We next ask whether a localized workload can be learned directly by a compact policy, or whether foundation representations remain necessary. We compare two compact Students (58M and 108M parameters), a 3.6B $\pi_{0.5}$ -architecture model trained without foundation pretraining, and a 3.05B diagnostic configuration that feeds frozen $\pi_{0.5}$ representations to a lightweight Student decoder. We report both validation loss and closed-loop task success.

As shown in Fig. 3(d), the 58M and 108M Students achieve only 6.0% and 7.4% success, respectively, while the 3.6B $\pi_{0.5}$ -Arch baseline reaches 16.8%. Increasing capacity therefore improves performance, but does not recover reliable closed-loop behavior.

Conditioning the lightweight decoder on frozen $\pi_{0.5}$ representations reduces validation loss to 0.0981 and raises task success to 33.2%, substantially outperforming even the 3.6B baseline. Thus, although the deployment workload is localized, learning it remains significantly easier in the foundation representation space.

Observation 3: Deployment locality does not eliminate the need for foundation knowledge: compact target-trained policies achieve only 6.0–7.4% success, versus 33.2% with frozen foundation representations.

Insight 3: Effective specialization should therefore transfer deployment-relevant foundation knowledge into a lightweight Student, rather than relearning the target policy from scratch. This motivates the Student Policy Distillation procedure in Sec. 4.3.

4 PICOVISTA DESIGN

The preceding observations motivate aggressive deployment-specific specialization: repetitive manipulation workloads occupy only a localized region of a foundation VLA’s representation space and need not incur its full generality tax at every decision. However, an ultra-compact on-device policy may discard capacity essential for handling ambiguous observations, contact-sensitive transitions, and off-nominal states, improving efficiency at the cost of robustness.

To address this tension, we propose PicoVISTA, an ultra-lightweight embodied intelligence framework that delegates nominal control to a compact policy and invokes additional reasoning only when its proposal is predicted to be unreliable. We next present the overall framework, followed by its policy distillation, predictive modeling, candidate evaluation, and selective intervention components.

4.1 Framework Overview

The overall pipeline of PicoVISTA is illustrated in Fig. 4. We first develop a compact stochastic Student policy to handle routine decisions efficiently and invoke additional evaluation only when its nominal proposal is predicted to be unreliable.

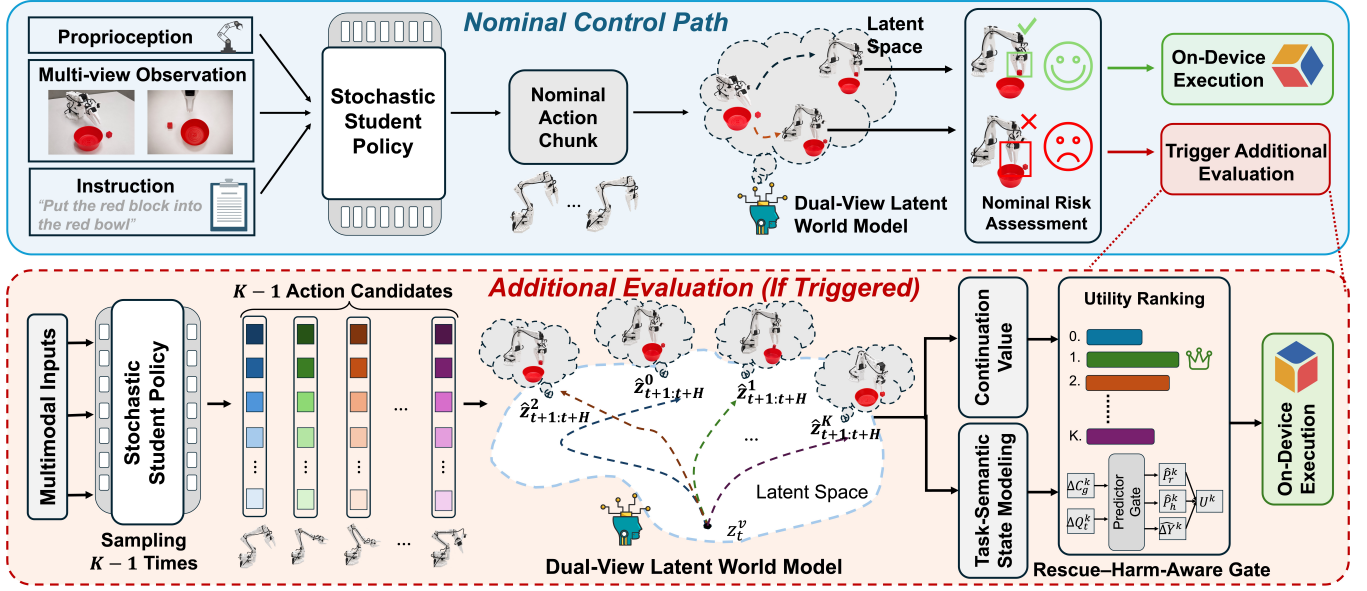


Figure 4: Overview pipeline of our proposed PicoVISTA.

In such cases, the system samples alternative actions from the Student, predicts their outcomes, and evaluates both short-horizon task progress and long-horizon continuation success. An alternative is executed only when the intervention is predicted to be beneficial and safe. In this way, PicoVISTA exploits deployment locality for aggressive model specialization while selectively recovering the reasoning capacity required for robust task execution.

At time t , the robot obtains the multi-modal information $s_t = (o_t^a, o_t^w, p_t, g)$, where o_t^a and o_t^w are the agent- and wrist-view observations, p_t is the proprioceptive state, and g denotes the task. The distilled stochastic Student π_θ samples a nominal action chunk

$$\mathbf{A}_t^0 = [\mathbf{a}_t^0, \dots, \mathbf{a}_{t+H_a-1}^0] \sim \pi_\theta(\cdot | s_t; \xi_0), \quad (1)$$

where ξ_0 is sampling randomness. Here, K denotes the total candidate budget, including the nominal proposal $\mathbf{A}_t^0$. If additional evaluation is triggered, the remaining $K-1$ alternatives are sampled from the same state:

$$\mathcal{A}_t = \{\mathbf{A}_t^0, \dots, \mathbf{A}_t^{K-1}\}, \quad \mathbf{A}_t^k \sim \pi_\theta(\cdot | s_t; \xi_k), \quad k = 1, \dots, K-1. \quad (2)$$

For each candidate, the dual-view latent world model predicts the consequences of its first $H \leq H_a$ actions. The resulting trajectory is evaluated by a semantic score $\mathbf{C}_g^k$, which represents short-horizon task progress, and an action-conditioned continuation value Q_t^k , which estimates eventual success after the candidate prefix is executed and control returns to the Student.

The nominal risk is $R_t = 1 - Q_t^0$. Low-risk nominal actions are executed directly; otherwise, alternatives are evaluated by semantic progress, continuation value, action deviation, and decision context. A rescue-harm-aware gate selects the highest-utility candidate k^* only if it is beneficial and safe; otherwise, the nominal chunk $\mathbf{A}_t^0$ is retained. The system

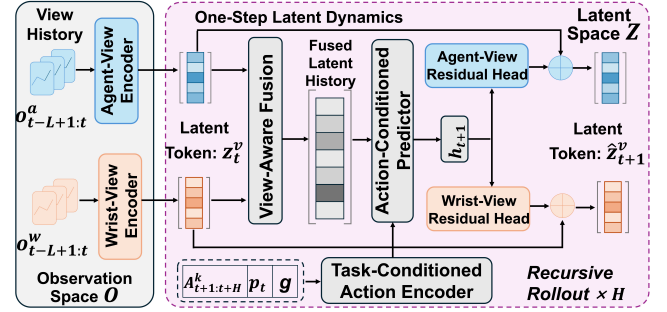


Figure 5: Dual-View latent world model for action-conditioned prediction.

executes the first H actions, reobserves, and replans. Unless stated otherwise, action chunks use physical control units.

4.2 Dual-View Latent World Model

To evaluate candidate actions before execution, PicoVISTA employs a dual-view latent WM $\mathcal{W}_\omega$ that predicts their short-horizon consequences in latent space. The agent view (a) captures global scene context, while the wrist view (w) provides local information about manipulation and contact. The two views are encoded separately, fused for shared action-conditioned dynamics prediction, and decoded by view-specific heads. For $v \in \{a, w\}$,

$$\mathbf{z}_t^v = P_v(E_v(o_t^v)), \quad \mathbf{f}_t = F_{\text{fuse}}(\text{Concat}_{v \in \{a, w\}} [\text{LN}_v(\mathbf{z}_t^v) + \mathbf{e}_v]). \quad (3)$$

A shared action-conditioned predictor models the dynamics:

$$\mathbf{h}_{t+1} = D(\mathbf{f}_{t-L+1:t}, \mathbf{c}_{t-L+1:t}), \quad \hat{\mathbf{z}}_{t+1}^v = \mathbf{z}_t^v + H_v(\mathbf{h}_{t+1}). \quad (4)$$

The model is first pretrained on generic interaction data and then adapted to task-specific multi-step prediction. Pre-training uses action conditioning $\mathbf{c}_t = E_A(\tilde{\mathbf{a}}_t)$, whereas task adaptation includes proprioception and task information:

$$\mathbf{c}_t = E_A(\tilde{\mathbf{a}}_t) + E_p(\tilde{\mathbf{p}}_t) + E_g(g). \quad (5)$$

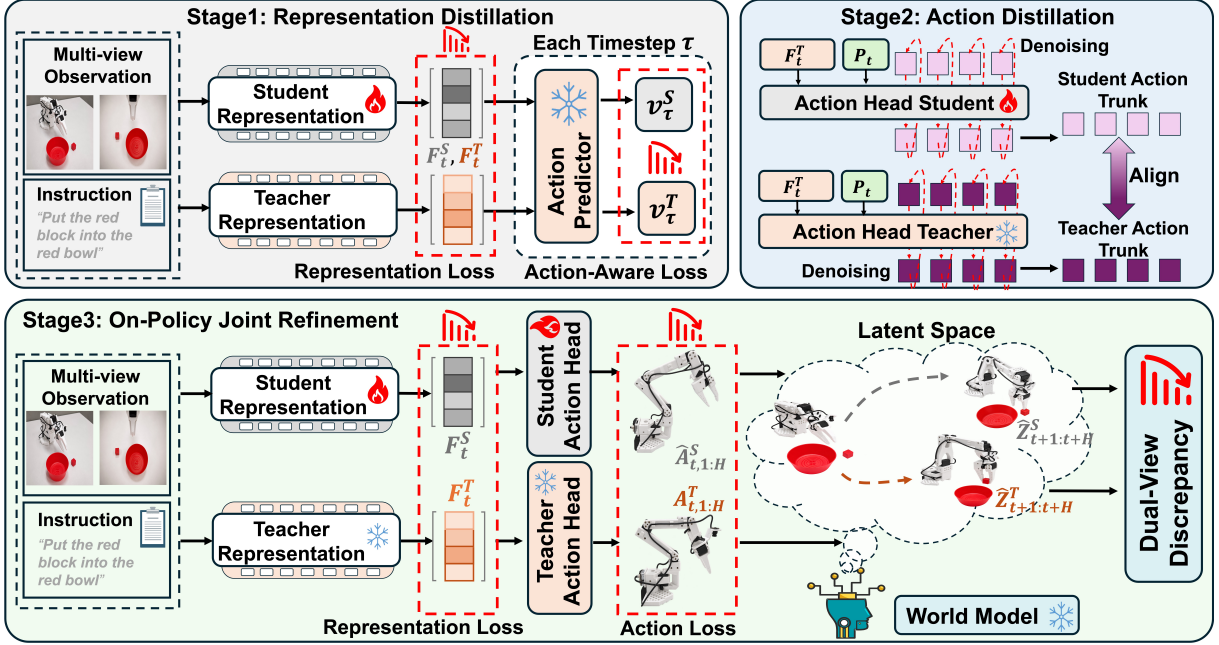


Figure 6: Pipeline of the world-model-guided three-stage student policy distillation framework.

The pretraining objective is

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{MSE}}^a + \lambda_w^{\text{pre}} \mathcal{L}_{\text{MSE}}^w + \lambda_{\text{reg}} \sum_{v \in \{a, w\}} \mathcal{L}_{\text{SIGReg}}^v. \quad (6)$$

During task adaptation, the visual encoders and projectors are frozen, and the remaining modules are optimized with

$$\mathcal{L}_{\text{task}} = \mathcal{L}_a + \lambda_w^{\text{task}} \mathcal{L}_w, \mathcal{L}_v = \mathcal{L}_{\text{feature}}^v + \lambda_{\Delta}^{\text{wm}} \mathcal{L}_{\Delta}^v + \lambda_{\text{temp}} \mathcal{L}_{\text{temporal}}^v. \quad (7)$$

For candidate action sequence $\mathbf{A}_t^k$, recursive rollout yields

$$\hat{\mathbf{Z}}_{t+1:t+H}^k = \mathcal{W}_{\omega} \left(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\mathbf{A}_{t,1:H}^k), \mathcal{N}_p(\mathbf{p}_t), g \right), \quad (8)$$

where $\mathbf{Z}_t = (z_t^a, z_t^w)$. At inference time, the model recursively predicts a dual-view latent trajectory for each candidate action sequence, with proprioceptive and task conditioning fixed at the rollout anchor. Implementation details are provided in Appendix A.1.1–A.1.4.

4.3 Student Policy Distillation

The stochastic Student π_{θ} serves as both the nominal controller and the proposal distribution for counterfactual actions. To compress a powerful Teacher π_T into this deployable policy, we adopt a three-stage distillation framework. The first two stages distill representations and actions on states provided by the Teacher or dataset. Since the deployed Student may visit different states, the third stage collects Student rollouts and queries the Teacher for corrective actions at Student-induced states. A frozen WM further aligns the predicted consequences of Student and Teacher actions, encouraging the Student to match both the Teacher’s immediate behavior and its future effects.

Stage 1: Representation Distillation. We first align the Student intermediate representation with that produced by the

Teacher. Given Teacher and Student representations F_t^T and F_t^S of the same input, we optimize

$$\mathcal{L}_{\text{rep}} = \lambda_{\text{feat}} \mathcal{L}_{\text{feat}} + \lambda_{\text{cos}} \mathcal{L}_{\text{cos}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}} + \lambda_{\text{proj}} \mathcal{L}_{\text{proj}} + \lambda_{\Delta}^{\text{rep}} \mathcal{L}_{\Delta}^{\text{rep}}, \quad (9)$$

which aligns feature values, directions, token relations, action projections, and temporal changes.

We then introduce action-aware supervision:

$$\begin{aligned} \mathcal{L}_{\text{aware}} &= \lambda_{\text{vel}} \mathcal{L}_{\text{vel-cons}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow-GT}}, \\ \mathcal{L}_{S1} &= \mathcal{L}_{\text{rep}} + \lambda_{\text{aware}} \mathcal{L}_{\text{aware}}. \end{aligned} \quad (10)$$

The two terms align Teacher- and Student-conditioned flow velocities and anchor them to dataset actions. The Teacher and action generator are frozen, so only the Student representation pathway is updated. Details are provided in Appendix A.2.1 and App. A.2.2.

Stage 2: Action Distillation. Using fixed Teacher visual-language tokens, we train only the Student action head:

$$\begin{aligned} \mathcal{L}_{\text{act}} &= \lambda_{\text{vel}} \mathcal{L}_{\text{vel-KD}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow-GT}} + \lambda_{\text{end}} \mathcal{L}_{\text{end-KD}} \\ &\quad + \lambda_{\text{action}} \mathcal{L}_{\text{action-GT}} + \lambda_{\text{grip}} \mathcal{L}_{\text{grip}}. \end{aligned} \quad (11)$$

This objective distills flow velocities and endpoints while supervising actions and gripper states with dataset targets. Critical steps are upweighted, including the first Student–dataset divergence, gripper transitions, and grasp events. The Student predicts the full action chunk rather than a residual. See App. A.2.3 for the implementation details.

Stage 3: On-policy Joint Refinement. We collect Student rollouts, query the Teacher at Student-induced states, and jointly refine the Student representation and action modules using on-policy targets and corrections. A frozen WM further aligns the predicted consequences of the differentiable

Student chunk $\hat{\mathbf{A}}_t^S$ and Teacher correction $\mathbf{A}_t^T$:

$$\hat{\mathbf{Z}}_{t+1:t+H}^S = \mathcal{W}_\omega \left(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\hat{\mathbf{A}}_{t,1:H}^S), \tilde{\mathbf{p}}_t, g \right), \quad (12)$$

$$\hat{\mathbf{Z}}_{t+1:t+H}^T = \text{sg} \left[\mathcal{W}_\omega \left(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\mathbf{A}_{t,1:H}^T), \tilde{\mathbf{p}}_t, g \right) \right]. \quad (13)$$

Their discounted dual-view discrepancy is

$$\mathcal{L}_{\text{WM}} = \frac{\sum_{h=1}^H \gamma^{h-1} [\ell_{\text{latent}}(\hat{\mathbf{z}}_{t+h}^{a,S}, \hat{\mathbf{z}}_{t+h}^{a,T}) + \lambda_{\text{w}}^{S3} \ell_{\text{latent}}(\hat{\mathbf{z}}_{t+h}^{w,S}, \hat{\mathbf{z}}_{t+h}^{w,T})]}{\sum_{h=1}^H \gamma^{h-1}}. \quad (14)$$

Detaching the Teacher branch allows future-aware gradients to update only the Student. The complete objective is

$$\mathcal{L}_{S3} = \mathcal{L}_{\text{rep}}^{\text{on}} + \lambda_{\text{act}} \mathcal{L}_{\text{act}}^{\text{on}} + \lambda_{\text{WM}} \mathcal{L}_{\text{WM}}. \quad (15)$$

Implementation details are described in Appendix A.2.4.

4.4 Task-Semantic State Modeling

Evaluating candidate actions requires identifying which task conditions they advance over a short horizon. Since the latent dynamics predicted by the WM are not directly interpretable, we introduce a task-semantic model that maps each observed-and-predicted trajectory to task-grounded predicates. These predicates capture both terminal conditions and events occurring during action execution.

Task-Grounded Semantic Predicates. For task g , define the semantic predicate vector

$$\Phi_g(s) = [\phi_1^g(s), \dots, \phi_{M_g}^g(s)] \in \{0, 1\}^{M_g}, \quad (16)$$

where each ϕ_j^g denotes a task-relevant physical condition grounded in the manipulation state. The task-specific predicates characterize meaningful progress, including proximity, grasp state, spatial relations, interaction state, and task completion. Labels are derived from environment states and task predicates in simulation, and from physical measurements or annotations in real-world data. For selected predicates, we additionally record continuous geometric margins near decision boundaries.

Dual-View Semantic Trajectory Representation. For candidate k , let y_j^k denote the trajectory-level label of predicate ϕ_j^g under a predefined endpoint-/event-based temporal rule, shared across simulated and real data. The model predicts

$$C_j^k = \sigma(\ell_j^k) \approx P(y_j^k = 1 \mid \mathbf{x}_{\text{sem}}^k), \quad \mathbf{C}_g^k = [C_1^k, \dots, C_{M_g}^k]. \quad (17)$$

For each view, the semantic input includes the observed and imagined trajectory and its temporal differences:

$$\mathbf{z}_{t-L+1:t+H}^{v,k} = [\mathbf{z}_{t-L+1:t}^v, \hat{\mathbf{z}}_{t+1:t+H}^{v,k}], \quad (18)$$

$$\Delta \mathbf{z}_\tau^{v,k} = \mathbf{z}_\tau^{v,k} - \mathbf{z}_{\tau-1}^{v,k}, \quad \tau = t-L+2, \dots, t+H, \quad (19)$$

where $v \in \{a, w\}$. We concatenate these features with proprioception, candidate actions, and the task embedding:

$$\begin{aligned} \mathbf{x}_{\text{sem}}^k &= \text{concat}(\mathbf{z}_{t-L+1:t+H}^{a,k}, \mathbf{z}_{t-L+1:t+H}^{w,k}, \Delta \mathbf{z}_{t-L+2:t+H}^{a,k}, \\ &\quad \Delta \mathbf{z}_{t-L+2:t+H}^{w,k}, \tilde{\mathbf{p}}_t, \tilde{\mathbf{A}}_{t,1:H}^k, E_g(g)), \quad (20) \\ \mathbf{r}_{\text{sem}}^k &= f_{\text{sem}}(\mathbf{x}_{\text{sem}}^k). \end{aligned}$$

Separate heads predict predicate logits and continuous geometric margins.

Semantic Learning Objective. We use class-balanced binary classification for predicates and Smooth L1 regression for geometric margins:

$$\mathcal{L}_{\text{pred}} = \frac{1}{M_g} \sum_{j=1}^{M_g} w_j \text{ BCEWithLogits}(\ell_j, y_j), \quad (21)$$

$$\mathcal{L}_{\text{margin}} = \text{SmoothL1}(\hat{\mathbf{m}}, \mathbf{m}), \quad (22)$$

$$\mathcal{L}_{\text{sem}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{margin}} \mathcal{L}_{\text{margin}}. \quad (23)$$

Training uses both realized and WM-predicted futures to reduce the distribution gap encountered during candidate-action evaluation. The WM and visual encoders remain frozen, while predicted trajectories are detached and cached. Additional details are provided in Appendix A.3.

4.5 Action-Conditioned Continuation Value

Short-term progress does not necessarily lead to task completion. We therefore learn a continuation-value model that estimates the probability of eventual success after executing a candidate action chunk and then following the deployed Student. The value is action-conditioned, allowing different candidates from the same state to receive different scores. It is estimated from the complete observed-and-predicted trajectory rather than only its endpoint.

Continuation Value Definition. For candidate chunk $\mathbf{A}_t$,

$$Q^{\pi_\theta}(s_t, \mathbf{A}_t) = P(Y = 1 \mid s_t, \text{execute } \mathbf{A}_{t,1:H}, \text{ then follow } \pi_\theta), \quad (24)$$

is defined, where Y denotes the terminal task outcome and $\mathbf{A}_{t,1:H}$ is the receding-horizon prefix used at deployment. Unlike a state-only value function, $Q^{\pi_\theta}(s_t, \mathbf{A}_t)$ explicitly captures how different candidate actions from the same state affect the probability of eventual success.

Trajectory-Aware Estimation. The WM predicts the candidate-conditioned future:

$$\begin{aligned} \hat{\mathbf{Z}}_{t+1:t+H}^A &= \mathcal{W}_\omega(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\mathbf{A}_{t,1:H}), \tilde{\mathbf{p}}_t, g), \\ \mathbf{X}_t^A &= [\mathbf{Z}_{t-L+1:t}, \hat{\mathbf{Z}}_{t+1:t+H}^A]. \end{aligned} \quad (25)$$

A lightweight encoder estimates the value logit, auxiliary progress, and a representation used by the intervention gate:

$$q_t^A = f_{\text{value}, \psi}(\mathbf{X}_t^A, \mathbf{p}_{t-L+1:t}), \quad \hat{p}_{t+H}^A = f_{\text{prog}}(\mathbf{X}_t^A, \mathbf{p}_{t-L+1:t}), \quad (26)$$

$$\mathbf{r}_t^A = f_{\text{repr}}(\mathbf{X}_t^A, \mathbf{p}_{t-L+1:t}), \quad Q_t^A = \sigma(q_t^A / T_{\text{cal}}). \quad (27)$$

Here, T_{cal} is fitted on held-out data, and Q_t^A is the calibrated continuation probability used by the risk trigger and intervention gate.

Continuation-Value Learning Objective. Each candidate is labeled by its continuation outcome:

$$Y_t^A = \mathbb{I}[\text{executing } \mathbf{A}_{t,1:H} \text{ and then following } \pi_\theta \text{ succeeds}]. \quad (28)$$

The primary objective is

$$\mathcal{L}_{\text{MC}} = \text{BCEWithLogits}(q_t^A, Y_t^A). \quad (29)$$

We further introduce progress prediction, candidate ranking, temporal consistency, and alignment between predicted and realized trajectories:

$$\mathcal{L}_Q = \lambda_{\text{MC}} \mathcal{L}_{\text{MC}} + \lambda_{\text{TD}} \mathcal{L}_{\text{TD}} + \lambda_{\text{prog}} \mathcal{L}_{\text{prog}} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}} + \lambda_{\text{align}} \mathcal{L}_{\text{align}}. \quad (30)$$

The frozen WM provides trajectory inputs but receives no gradients from this objective. See App. A.4 for details.

4.6 Risk-Aware Intervention

To reduce on-device computation, we evaluate additional action candidates only when necessary. If the nominal action has a high predicted success probability, it is executed directly; otherwise, a more expensive candidate comparison is triggered. This selective mechanism concentrates computation on high-risk decisions.

Risk-Triggered Reasoning. The nominal candidate is evaluated from its predicted trajectory:

$$Q_t^0 = Q_\psi \left([Z_{t-L+1:t}, \hat{Z}_{t+1:t+H}^0], \mathbf{p}_{t-L+1:t} \right), \quad R_t = 1 - Q_t^0. \quad (31)$$

Additional candidates are evaluated only if $R_t \geq \tau_{\text{risk}}$. The scheduler also enforces the phase-specific candidate budgets described in Appendix A.5.

Candidate Evaluation. If triggered, the Student generates

$$\mathcal{A}_t = \{\mathbf{A}_t^0, \mathbf{A}_t^1, \dots, \mathbf{A}_t^{K-1}\}, \quad \mathbf{A}_t^k \sim \pi_\theta(\cdot \mid s_t; \xi_k), \quad (32)$$

where candidates share the same context but use different sampling noise. For each candidate, the world model predicts

$$\hat{Z}_{t+1:t+H}^k = \mathcal{W}_\omega \left(Z_{t-L+1:t}, \mathcal{N}_A(\mathbf{A}_{t,1:H}^k), \tilde{\mathbf{p}}_t, g \right). \quad (33)$$

The semantic and value models produce $\mathbf{C}_g^k$ and Q_t^k , with nominal-relative gains

$$\Delta \mathbf{C}_g^k = \mathbf{C}_g^k - \mathbf{C}_g^0, \quad \Delta Q_t^k = Q_t^k - Q_t^0. \quad (34)$$

The signals remain separate, as short-term semantic progress may not imply eventual success. Non-finite or overly deviating candidates are filtered before learned selection.

Rescue-Harm-Aware Gate. Using semantic, value, action-deviation, and decision-context features, the gate predicts

$$\left(\hat{p}_{\text{rescue}}^k, \hat{p}_{\text{harm}}^k, \widehat{\Delta Y}^k \right). \quad (35)$$

For paired terminal outcomes $y_0, y_k \in \{0, 1\}$, the targets are

$$y_{\text{rescue}} = (1 - y_0)y_k, \quad y_{\text{harm}} = y_0(1 - y_k), \quad y_{\text{gain}} = y_k - y_0. \quad (36)$$

Auxiliary success heads regularize training without adding deployment thresholds. Eligible candidates are ranked by

$$U^k = \hat{p}_{\text{rescue}}^k - \alpha_{\text{harm}} \hat{p}_{\text{harm}}^k + \alpha_{\text{gain}} \widehat{\Delta Y}^k, \quad (37)$$

subject to

$$\hat{p}_{\text{rescue}}^k \geq \tau_{\text{rescue}}, \quad \hat{p}_{\text{harm}}^k \leq \tau_{\text{harm}}, \quad \widehat{\Delta Y}^k \geq \tau_{\text{gain}}. \quad (38)$$

The gate abstains if no candidate satisfies all constraints.

Receding-Horizon Execution. Let k^* denote the feasible candidate with the highest utility. If k^* is accepted, the system executes $\mathbf{A}_t^{k^*}$; otherwise, it executes the nominal chunk $\mathbf{A}_t^0$. The system then executes the first H actions, reobserves the environment, and replans. Gate training, calibration, safety filters, and budgeting are detailed in App. A.5.

4.7 Online Decision Procedure

We integrate the preceding modules into an online decision procedure, summarized in Algorithm 1. The system executes the nominal Student action when its predicted risk is low; otherwise, it evaluates alternative candidates and selects a safe, beneficial replacement.

Algorithm 1 Risk-Aware Online Inference

Require: Student π_θ , world model $\mathcal{W}_\omega$, semantic model f_{sem} , value model Q_ψ , gate G , threshold τ_{risk} , total candidate budget K

- 1: Observe s_t and encode $\mathbf{Z}_{t-L+1:t}$ and $\mathbf{p}_{t-L+1:t}$
- 2: Sample nominal action $\mathbf{A}_t^0 \sim \pi_\theta(\cdot \mid s_t; \xi_0)$
- 3: Predict $\hat{\mathbf{Z}}_{t+1:t+H}^0$ and evaluate Q_t^0
- 4: **if** $1 - Q_t^0 < \tau_{\text{risk}}$ or the phase budget is exhausted **then**
- 5: Execute $\mathbf{A}_{t,1:H}^0$ and **return**
- 6: **end if**
- 7: Evaluate nominal semantics $\mathbf{C}_g^0$
- 8: Sample alternatives $\mathbf{A}_t^k \sim \pi_\theta(\cdot \mid s_t; \xi_k)$, $k = 1, \dots, K - 1$
- 9: **for** $k = 1, \dots, K - 1$ **do**
- 10: Predict $\hat{\mathbf{Z}}_{t+1:t+H}^k$
- 11: Evaluate $\mathbf{C}_g^k$, Q_t^k , $\Delta \mathbf{C}_g^k$, and ΔQ_t^k
- 12: Apply deterministic safety constraints
- 13: **end for**
- 14: Score eligible candidates using G
- 15: **if** any candidate satisfies all gate constraints **then**
- 16: Select $k^* = \arg \max_k U^k$ and execute $\mathbf{A}_{t,1:H}^{k^*}$
- 17: **else**
- 18: Execute $\mathbf{A}_{t,1:H}^0$
- 19: **end if**
- 20: Reobserve and replan

QNN compilation and DLC deployment. We export all neural modules as hardware-agnostic QNN Deep Learning Container artifacts and execute them on-device through the Qualcomm AI Engine Direct runtime [40]. The robot maintains synchronized observation buffers and locally performs candidate sampling, model inference, and receding-horizon control. Backend selection, numerical precision, operator coverage, and closed-loop latency are reported in the deployment evaluation, as successful compilation alone does not establish runtime performance.

5 EVALUATION

5.1 Experimental Setup

Training setup. All models are implemented in PyTorch and trained with Adam on a single NVIDIA H20 GPU. We use



Figure 7: Multi-view dual-arm robotic platform setup (SO-ARM101 $\times$ 2).

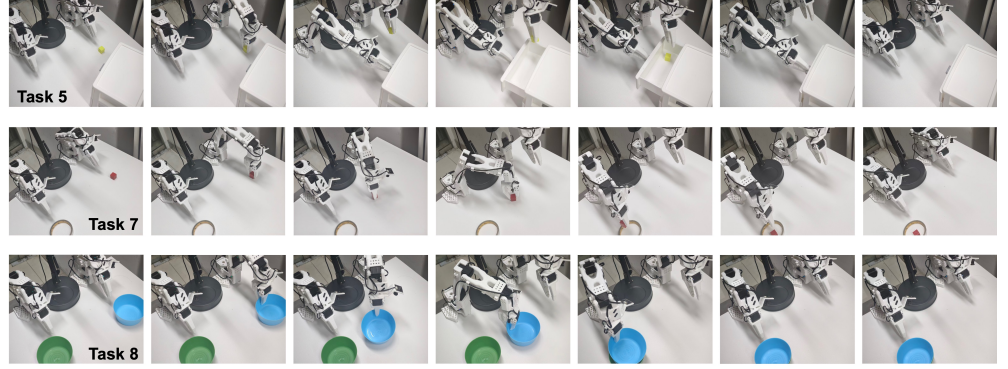


Figure 8: Execution progress of three example dual-arm manipulation tasks. Task 5: Put the yellow block into the drawer; Task 7: Put the red block inside the circle; Task 8: Stack the blue bowl on top of the green bowl. All real-world tasks are in Tab. 2.

Table 2: Real-world manipulation tasks.

Task ID	Type	Language instruction
1	Single-arm	Put the red block into the red bowl.
2	Single-arm	Put the red block inside the circle.
3	Single-arm	Stack the red block on top of the blue block.
4	Single-arm	Stack the blue bowl on top of the green bowl.
5	Bimanual	Put the yellow block into the drawer.
6	Bimanual	Pass the green bowl from the left hand to the right hand.
7	Bimanual	Put the red block inside the circle.
8	Bimanual	Stack the blue bowl on top of the green bowl.

$\pi_{0.5}$ as the foundation Teacher and distill it into the stochastic Student described in Sec. 4.3.

Simulation setup. We evaluate on LIBERO [39], CALVIN [38], and the WidowX setting of SimplerEnv. LIBERO contains 40 tasks across the Spatial, Object, Goal, and Long suites and serves as our primary benchmark. For CALVIN, we follow the standard ABC $\rightarrow$ D protocol, training on environments A–C and evaluating 1,000 five-task instruction chains in environment D. SimplerEnv evaluation covers four WidowX manipulation tasks: spoon placement, carrot placement, object stacking, and eggplant placement. We follow the native observation, action, and task definitions of each benchmark.

Real-world setup. We build single-arm and bimanual platforms using the open-source SO-ARM101 manipulator. The single-arm setup uses a fixed front-view camera, whereas the bimanual setup uses an overhead camera; both include wrist-mounted cameras for local observations. RGB streams are captured at 640×480 and 30 fps. Each arm has six-dimensional joint states and actions, yielding 12-dimensional representations for the bimanual setup. Fig. 7 shows the bimanual platform, which is similar to the single-arm setup.

Real-world tasks. We evaluate four single-arm and four bimanual tasks (Tab. 2). For each task, we collect 1,500 teleoperated demonstrations from 300 randomized initial configurations to fine-tune the $\pi_{0.5}$ Teacher. Data are stored in LeRobot v3.0 format with aligned RGB observations, proprioception, actions, and language instructions.

Edge hardware platforms. We use a Rubik Pi 3 with a Qualcomm QCS6490 SoC for closed-loop control. As a low-cost, resource-constrained edge platform (only CNY799 and US\$179), the Rubik Pi 3 represents a substantially more limited deployment target than flagship smartphones and PC-class reference designs. Its Hexagon 770 NPU provides up to 12 TOPS. The platform has 8 GB LPDDR4x memory and 128 GB UFS 2.2 storage. We also benchmark on four other edge devices: the Samsung Galaxy S24 (CNY5499) and S25 (CNY5999), Snapdragon 8 Elite QRD (US\$1499), and Snapdragon X Elite CRD (US\$899). Neural modules are exported as QNN Deep Learning Container artifacts and executed locally via Qualcomm AI Engine Direct.

Evaluation metric. Task success rate is the primary metric for both simulation and real-world experiments. A rollout succeeds only if the task-specific completion condition is satisfied within the execution horizon. For CALVIN, we additionally report the fraction of sequences completing at least one through five consecutive tasks.

5.2 Overall Performance

We compare PicoVISTA with representative VLA policies on three simulation benchmarks and eight real-world tasks. We report task performance and total inference-time parameters to assess control capability and deployability.

CALVIN. As shown in Tab. 3, PicoVISTA completes all five consecutive tasks in 62.3% of sequences, with an average sequence length of 4.10. It matches OpenVLA-OFT with approximately $99\times$ fewer inference-time parameters and outperforms MoDE and Seer. Compared with the 3.62B-parameter $\pi_{0.5}$ Teacher, it uses $46.4\times$ fewer parameters with only a 0.51 sequence-length gap, demonstrating strong language-conditioned long-horizon control with a compact deployment footprint.

SimplerEnv WidowX. Tab. 4 shows that PicoVISTA achieves 87.5% overall success, with task-level rates of 79.2%–95.8%.

¹Best, second-best, and third-best results are shown in bold, underlined, and shaded, respectively. Tied results share the same ranking and formatting.

Table 3: CALVIN ABC→D results. Columns 1–5 report the percentage of sequences completing at least 1–5 consecutive tasks.¹

Method	Full Para.	Tasks completed in a row $\uparrow$					Avg. len. $\uparrow$
		1	2	3	4	5	
Large models ($P \geq 4$ B)							
OpenVLA [1]	≈ 7.54 B	91.3	77.8	62.0	52.1	43.5	3.27
OpenVLA-OFT [22]	≈ 7.71 B	<u>96.3</u>	89.1	82.4	75.8	<u>66.5</u>	<u>4.10</u>
VLAS [41]	> 7.063 B	87.2	64.2	40.9	28.1	19.6	2.40
LCB [42]	> 7.063 B	73.6	50.2	28.5	16.0	9.9	1.78
Small models ($1 \text{ B} \leq P < 4 \text{ B}$)							
DeeR [43]	> 3.00 B	86.2	70.1	51.8	41.5	30.4	2.82
RoboFlamingo [44]	> 3.00 B	82.4	61.9	46.6	33.1	23.5	2.48
SuSIE [45]	≈ 1.30 B	87.0	69.0	49.0	38.0	26.0	2.69
$\pi_{0.5}$ [2]	≈ 3.62 B	99.6	96.7	92.5	89.4	82.8	4.61
Tiny models ($P < 1 \text{ B}$)							
Seer-Large [46]	≈ 0.566 B	<u>96.3</u>	<u>91.6</u>	<u>86.1</u>	<u>80.3</u>	<u>74.0</u>	<u>4.28</u>
MoDE [47]	≈ 0.740 B	<u>96.2</u>	88.9	81.1	71.8	63.5	4.01
Seer [46]	≈ 0.316 B	94.4	87.2	79.9	72.2	64.3	3.98
PicoVISTA (Ours)	≈ 0.078 B	<u>96.2</u>	<u>91.1</u>	<u>83.4</u>	<u>76.5</u>	62.3	<u>4.10</u>

Table 4: Comparison on SimplerEnv WidowX robot tasks.¹

Method	Full Para.	Spoon	Carrot	Stack	Eggplant	Avg.
Large models ($P \geq 4$ B)						
OpenVLA [1]	≈ 7.54 B	0.0	0.0	0.0	4.1	1.0
OpenVLA-OFT [22]	≈ 7.71 B	12.5	4.2	8.3	37.5	15.6
SpatialVLA [48]	≈ 4.03 B	16.7	25.0	29.2	100.0	42.7
MemoryVLA [49]	≈ 8.38 B	<u>75.0</u>	<u>75.0</u>	37.5	100.0	71.9
Small models ($1 \text{ B} \leq P < 4 \text{ B}$)						
π_0 [9]	≈ 3.50 B	29.1	0.0	16.7	62.5	27.1
π_0 -FAST [14]	≈ 2.92 B	29.1	21.9	10.8	66.6	32.1
GR00T-N1 [10]	≈ 2.19 B	62.5	<u>45.8</u>	16.7	20.8	36.5
FLOWER [50]	≈ 1.00 B	71.0	13.0	8.0	<u>88.0</u>	45.0
Tiny models ($P < 1 \text{ B}$)						
X-VLA [3]	≈ 0.88 B	100.0	91.7	95.8	<u>95.8</u>	95.8
SimVLA [4]	≈ 0.81 B	100.0	91.7	<u>91.7</u>	100.0	95.8
PicoVISTA (Ours)	≈ 0.078 B	<u>95.8</u>	91.7	<u>83.3</u>	79.2	<u>87.5</u>

Only X-VLA and SimVLA perform better, but use $11.3\times$ and $10.4\times$ more parameters, respectively. PicoVISTA also surpasses MemoryVLA, the strongest large-model baseline, by 15.6 percentage points, demonstrating competitive cross-scene manipulation without billion-scale inference.

LIBERO. As shown in Tab. 5, PicoVISTA achieves 96.4% average success across four LIBERO suites, including 98.4% on Spatial and 99.2% on Object. It trails the 3.62B-parameter $\pi_{0.5}$ Teacher by only 0.5 points while using $46.4\times$ fewer parameters, and outperforms OpenVLA and Octo by 19.9 and 21.3 points. SimVLA and X-VLA score 2.2 and 1.7 points higher but require over $10\times$ more parameters, demonstrating PicoVISTA’s competitive multi-task performance at a much smaller scale.

Table 5: LIBERO success rates (%) and audited inference-time parameter counts.¹

Method	Full Para.	Spatial	Object	Goal	Long	Avg.
Large models ($P \geq 4 \text{ B}$)						
OpenVLA-OFT [22]	≈ 7.71 B	97.6	98.4	97.9	94.5	97.1
MemoryVLA [49]	≈ 8.38 B	<u>98.4</u>	98.4	96.4	93.4	96.7
UnifiedVLA [51]	≈ 8.80 B	95.4	98.8	93.6	94.0	95.5
OpenVLA [1]	≈ 7.54 B	84.7	88.4	79.2	53.7	76.5
Small models ($1 \text{ B} \leq P < 4 \text{ B}$)						
VLA-Adapter-Pro [52]	≈ 1.47 B	99.6	<u>99.6</u>	<u>98.2</u>	<u>96.4</u>	<u>98.5</u>
VLA-Adapter [52]	≈ 1.36 B	97.8	<u>99.2</u>	<u>97.2</u>	<u>95.0</u>	97.3
$\pi_{0.5}$ [2]	≈ 3.62 B	<u>98.8</u>	98.2	<u>98.0</u>	92.4	96.9
FLOWER [50]	≈ 1.00 B	97.1	96.7	95.6	93.5	95.7
Tiny models ($P < 1 \text{ B}$)						
SimVLA [4]	≈ 0.81 B	99.6	99.8	98.6	96.4	98.6
X-VLA [3]	≈ 0.88 B	98.2	98.6	97.8	97.6	<u>98.1</u>
Octo [17]	≈ 0.203 B	78.9	85.7	84.6	51.1	75.1
PicoVISTA (Ours)	≈ 0.078 B	<u>98.4</u>	<u>99.2</u>	94.6	93.2	96.4

Table 6: Real-world success rates (%). Tasks 1–4 are single-arm and Tasks 5–8 are bimanual, ordered as in Tab. 2.

Method	Single-arm tasks				Bimanual tasks				Avg.
	1	2	3	4	5	6	7	8	
$\pi_{0.5}$ [2] ($\approx 3.62 \text{ B}$)	98	98	95	96	94	94	96	96	95.9
X-VLA [3] ($\approx 0.88 \text{ B}$)	92	93	94	94	95	93	94	92	93.4
PicoVISTA ($\approx 0.078 \text{ B}$)	91	92	90	92	88	89	89	87	89.8

Real-world manipulation. Tab. 6 shows that PicoVISTA achieves 89.8% average success across eight physical tasks: 91.3% on single-arm and 88.3% on bimanual tasks, with all tasks above 87%. It trails X-VLA and the $\pi_{0.5}$ Teacher by only 3.6 and 6.1 points while using $11.3\times$ and $46.4\times$ fewer parameters, demonstrating consistent on-device performance across both settings.

5.3 On-Device Performance

We benchmark four inference cases on five edge devices as described in Sec. 5.1. The cases include (1) the standalone PicoVISTA Student; (2) triggered Risk-Aware Intervention with $K = 8$ chunks (one nominal and seven alternatives); (3) VLA-Adapter [52]; and (4) VLA-Adapter-Pro. Under the same QNN setting, we measure end-to-end decision latency and peak memory. The triggered result represents the costly intervention path rather than amortized runtime.

Fig. 9a shows that the standalone Student takes 5.65–7.21 ms on the four higher-end platforms excluding Rubik Pi 3, versus 47.80–69.58 ms for triggered Risk-Aware Intervention. On Rubik Pi 3, they take 323.19 and 2522.15 ms, respectively. Despite evaluating seven additional alternatives, the triggered path remains 2.5–3.8 $\times$ faster than VLA-Adapter and 2.7–3.9 $\times$ faster than VLA-Adapter-Pro across all devices. As shown in Fig. 9b, the triggered path uses

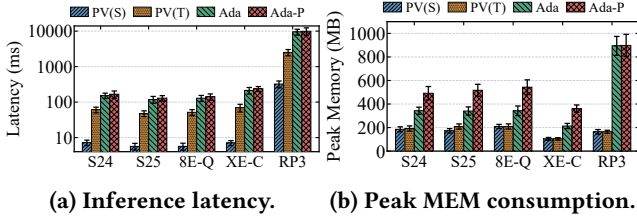


Figure 9: On-device per-decision cost (log scale) across five platforms. PV(S): Student; PV(T): triggered intervention with $K = 8$ actions; Ada/Ada-P: VLA-Adapter/Pro; 8E-Q/XE-C/RP3: Snapdragon 8 Elite QRD/X Elite CRD/Rubik Pi 3.

Table 7: Effects of Stage 3 (S3) on-policy joint refinement, $\mathcal{L}_{WM}$, and Risk-Aware Intervention (RAI) on LIBERO task success (%).

Variant	S3	$\mathcal{L}_{WM}$	RAI	LIBERO suite				Avg.
				Spatial	Object	Goal	Long	
S2 Student	✗	✗	✗	82.8	86.6	78.4	72.2	80.0
S3 w/o $\mathcal{L}_{WM}$	✓	✗	✗	91.6	92.8	87.4	85.2	89.3
S3 w/ $\mathcal{L}_{WM}$	✓	✓	✗	92.6	93.2	88.6	86.0	90.1
S3 w/o $\mathcal{L}_{WM}$ + RAI	✓	✗	✓	96.8	98.4	94.2	93.0	95.6
PicoVISTA (full)	✓	✓	✓	98.4	99.2	94.6	93.2	96.4

103.59–209.54 MB of peak memory, 38.7–81.7% less than VLA-Adapter and 59.6–81.8% less than VLA-Adapter-Pro. Thus, PicoVISTA retains substantial on-device efficiency even under full candidate evaluation.

5.4 Ablation Studies

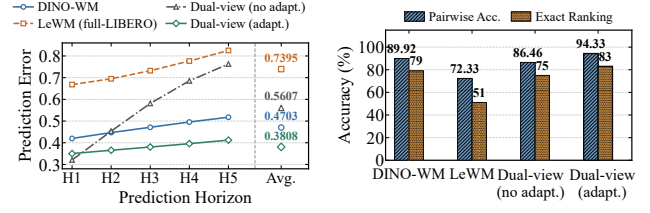
We ablate the three-stage Student Policy Distillation pipeline (Sec. 4.3), the Dual-View Latent World Model (Sec. 4.2), and Risk-Aware Intervention (Sec. 4.6). The corresponding online decision procedure is summarized in Algorithm 1.

5.4.1 Student Policy Distillation Ablation. We first isolate Stage 3 on-policy refinement, its frozen-WM loss $\mathcal{L}_{WM}$, and Risk-Aware Intervention. All Stage 3 variants share the same rollouts and optimization settings. Removing $\mathcal{L}_{WM}$ only sets $\lambda_{WM} = 0$, retaining the representation and action objectives in Eq. 15. For Risk-Aware Intervention, the Student is frozen to separate training-time distillation from online candidate selection. Tab. 7 shows that Stage 3 on-policy refinement without $\mathcal{L}_{WM}$ raises average success from 80.0% to 89.3%, the largest distillation gain, including 13.0 points on LIBERO-Long. Adding $\mathcal{L}_{WM}$ improves all suites and increases the average to 90.1%, indicating that dual-view consequence alignment provides supervision beyond representation and action objectives.

Risk-Aware Intervention further improves both Stage 3 Students: from 89.3% to 95.6% without $\mathcal{L}_{WM}$ and from 90.1% to 96.4% with it. Combining a stronger nominal Student with risk-triggered candidate evaluation yields the best results across all four suites.

5.4.2 Dual-View Latent World Model Ablation.

Prediction quality. We first assess whether task adaptation improves recursive prediction. Given a 3-step latent history $Z_{t-2:t}$ and aligned executed actions, each WM recursively



(a) Recursive canonical-state prediction error. (b) Native-latent similarity-order preservation.

Figure 10: Recursive prediction on held-out LIBERO-Spatial trajectories: (a) canonical-state NRMSE over H1–H5 and the mean; (b) pairwise and exact ranking accuracy.

Table 8: LIBERO success rates (%) when Risk-Aware Intervention uses different WMs. W/o TA means without task adaptation.

WM	Spatial	Object	Goal	Long	Avg.
LeWM	92.2	93.4	89.2	85.6	90.1
DINO-WM	96.8	97.6	92.4	91.2	94.5
Dual-view latent WM w/o TA	94.0	94.4	90.2	89.8	92.1
Dual-view latent WM	98.4	99.2	94.6	93.2	96.4

predicts five future states without teacher forcing. We evaluate canonical-state NRMSE on approximately 664 windows from ten held-out LIBERO-Spatial trajectories, using frozen linear probes to map predictions to the same 8D robot state. On 100 windows, pairwise and exact-ranking accuracy assess whether predicted futures preserve real-representation similarity orderings. Since each model uses its own frozen encoder space, these metrics measure rollout consistency rather than directly compare latent spaces.

As shown in Fig. 10, without task adaptation, the dual-view latent WM has lower one-step error but drifts from 0.3207 at H1 to 0.7629 at H5. Task adaptation limits this increase to 0.3502–0.4122, yielding the lowest H1–H5 average (0.3808). It also improves pairwise and exact-ranking accuracy from 86.46% to 94.33% and 75% to 83%, respectively, demonstrating better multi-step stability and latent similarity preservation.

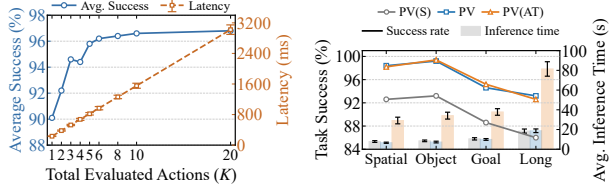
Closed-loop impact. We next evaluate Risk-Aware Intervention with different WMs. All variants share the same frozen Student, candidate sampling, rollout horizon, candidate budget, and risk trigger. Their task-semantic and continuation-value models use identical data splits, architectures, and training settings.

The task-adapted dual-view latent WM raises average success from 92.1% to 96.4%, outperforming its non-adapted counterpart, DINO-WM, and LeWM by 4.3, 1.9, and 6.3 points, respectively. Combined with the prediction results, this demonstrates that stable task-adapted rollouts improve candidate evaluation and closed-loop control.

5.4.3 Risk-Aware Intervention Ablation. We examine the gate inputs and candidate budget of Risk-Aware Intervention (Sec. 4.6), summarized in Alg. 1.

Table 9: Effects of the task-semantic state and action-conditioned continuation value supplied to the rescue-harm-aware gate. TS means task-semantic and CV means continuation value.

Setting	TS state	CV	Gate G	LIBERO suite				Avg.
				Spatial	Object	Goal	Long	
Student only	✗	✗	✗	92.6	93.2	88.6	86.0	90.1
TS only	✓	✗	✓	97.0	98.4	93.8	92.4	95.4
CV only	✗	✓	✓	97.8	96.6	93.2	92.6	95.1
PicoVISTA (full)	✓	✓	✓	98.4	99.2	94.6	93.2	96.4

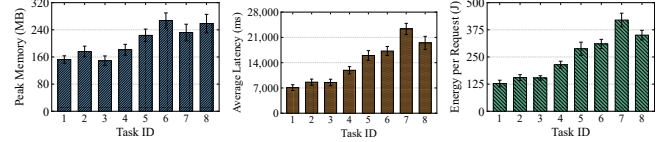


(a) Candidate-budget trade-off. (b) Risk-trigger effectiveness. Figure 11: Risk-Aware Intervention ablations on LIBERO. (a) Success and latency versus total candidate budget K . (b) Success and mean rollout inference time across trigger policies. PV(S), PV, and PV(AT) denote Student-only, selective-trigger, and Always-Trigger ($K = 8$), respectively.

Task-semantic state and action-conditioned continuation value. We ablate the task-semantic and action-conditioned continuation-value inputs to the rescue-harm-aware gate. All variants share the same frozen Student, dual-view latent WM, candidate sampling, rollout horizon, risk trigger, safety constraints, action-deviation features, and decision context. Removing continuation-value inputs does not remove Q_t^0 , which still defines the shared risk trigger in Eq. 31. Tab. 9 shows that supplying only the task-semantic state or only the action-conditioned continuation value raises average success from 90.1% to 95.4% and 95.1%, respectively. Combining both reaches 96.4%, outperforming the two single-signal variants by 1.0 and 1.3 percentage points and producing the best result on every suite. Thus, short-horizon task progress and long-horizon continuation success provide complementary evidence to the rescue-harm-aware gate.

Candidate-budget trade-off. We vary the total candidate budget K , including the nominal chunk A_t^0 . Thus, $K = 1$ evaluates only the nominal action, while triggered settings sample $K - 1$ alternatives. All other Risk-Aware Intervention components remain fixed. As shown in Fig. 11a, increasing K from 1 to 8 raises average success from 90.1% to 96.4%, after which gains saturate. While $K = 10$ and $K = 20$ achieve 96.6% and 96.8%, latency increases from 1261 ms at $K = 8$ to 1554 ms and 3022 ms. Thus, $K = 8$ offers the best trade-off, remaining within 0.4 points of $K = 20$ while reducing latency by 58.3%.

Risk-trigger effectiveness. We isolate Risk-Triggered Reasoning by comparing the same frozen Student under three modes: *Student only*, which executes the nominal action; *PicoVISTA*, which invokes Risk-Aware Intervention when



(a) Peak inference-time memory. (b) Cumulative inference latency. (c) Inference energy per task. Figure 12: Task-level resource cost of PicoVISTA on the eight real-world manipulation tasks.

$R_t \geq \tau_{\text{risk}}$; and *Always-Trigger*, which evaluates $K = 8$ chunks (one nominal and seven alternatives) at every decision. Both intervention modes share all other components, isolating the trigger policy. Fig. 11b shows that selective triggering improves average success from 90.1% to 96.4%. *Always-Trigger* also achieves 96.4%, differing by at most 0.6 points per suite, but requires 45.61 s of cumulative inference versus 10.65 s for PicoVISTA. Thus, selective triggering matches its accuracy while reducing inference time by 76.7%, remaining close to the 11.35 s Student-only baseline.

5.5 Real-World Task-Level Resource Cost

We finally profile the complete PicoVISTA system on Rubik Pi 3 across the eight physical tasks in Table 2. Task IDs 1–8 follow the row order in the table: Tasks 1–4 are single-arm tasks and Tasks 5–8 are bimanual tasks. For each task, we report peak inference-time memory, average cumulative neural-inference latency per rollout, and inference energy per task request. The latency includes only model computation and excludes image capture, communication, robot motion, and environment waiting time.

Fig. 12 shows that peak memory remains between 148.84 and 267.44 MB across all tasks, with Task 6 reaching the maximum. Average cumulative inference latency ranges from 7.10 to 23.33 s, while energy per task request ranges from 127.72 to 419.99 J; Task 7 has the highest value for both metrics. Averaged over Tasks 1–4, the single-arm workloads require 164.67 MB, 9.03 s, and 162.52 J, respectively. The corresponding averages for Tasks 5–8 are 245.34 MB, 19.00 s, and 342.07 J. Thus, the bimanual workloads use 49.0% more peak memory and approximately $2.1\times$ the cumulative inference time and energy, reflecting their longer inference sequences. Nevertheless, PicoVISTA completes all eight physical workloads with a peak memory footprint below 268 MB.

6 CONCLUSION

We presented PicoVISTA, an ultra-lightweight on-device framework that addresses the mismatch between the broad capacity of foundation VLAs and the concentrated workloads of fixed robotic deployments. By combining compact policy specialization with selective risk-aware intervention, PicoVISTA retains foundation knowledge while enabling efficient and reliable control on edge hardware. Experiments validate its effectiveness in simulated and real-world single- and bimanual tasks. Future work will explore broader embodiments, open-ended environments, continual adaptation, and robustness to uncertainty and irreversible failures.

REFERENCES

- [1] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [2] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [3] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 60580–60606, 2026.
- [4] Yuankai Luo, Woping Chen, Tong Liang, Baiqiao Wang, and Zhen-guo Li. Simvla: A simple vla baseline for robotic manipulation. *arXiv preprint arXiv:2602.18224*, 2026.
- [5] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [6] Xi Chen, Josip Djolonga, Piotr Padlewski, Basil Mustafa, Soravit Changpinyo, Jialin Wu, Carlos Riquelme Ruiz, Sebastian Goodman, Xiao Wang, Yi Tay, et al. Pali-x: On scaling up a multilingual vision and language model. *arXiv preprint arXiv:2305.18565*, 2023.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [8] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025.
- [9] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [10] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [11] Figure AI. Helix: A vision-language-action model for generalist humanoid control, February 2025. Technical report / online release.
- [12] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [13] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [14] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [15] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-vla: A 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.
- [16] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- [17] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [18] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. Vima: Robot manipulation with multimodal prompts. 2023.
- [19] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [20] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10(4):3988–3995, 2025.
- [21] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Yuxin Huang, Han Zhao, Donglin Wang, and Haoang Li. Ceed-vla: Consistency vision-language-action model with early-exit decoding. *arXiv preprint arXiv:2506.13725*, 2025.
- [22] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [23] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. *arXiv preprint arXiv:2410.04640*, 2024.
- [24] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. *arXiv preprint arXiv:2503.08558*, 2025.
- [25] Annie Xie, Fahim Tajwar, Archit Sharma, and Chelsea Finn. When to ask for help: Proactive interventions in autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 35:16918–16930, 2022.
- [26] Huihan Liu, Shivin Dass, Roberto Martín-Martín, and Yuke Zhu. Model-based runtime monitoring with interactive imitation learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4154–4161. IEEE, 2024.
- [27] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando De Freitas, and Serkan Cabi. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.
- [28] Rohan Sinha, Amine Elhafi, Christopher Agia, Matthew Foutter, Edward Schmerling, and Marco Pavone. Real-time anomaly detection and reactive planning with large language models. *arXiv preprint arXiv:2407.08735*, 2024.
- [29] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12124–12131. IEEE, 2024.
- [30] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.
- [31] Md Sadman Sakib and Yu Sun. Star: A foundation model-driven framework for robust task planning and failure recovery in robotic systems. *International Journal of Artificial Intelligence and Robotics Research*, 2:2550007, 2025.
- [32] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1702–1713. IEEE, 2025.

- [33] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohov, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [34] Jiahang Tu, Fengyu Yang, Chenyang Ma, Xihang Yu, Ziyao Zeng, Shaokai Wu, Hanbin Zhao, Zhi Tao, Chao Zhang, Hui Qian, et al. Unita: A unified multimodal model for cross-sensor tactile understanding and generation. *arXiv preprint arXiv:2606.31451*, 2026.
- [35] Kailin Wang, Haoxiang Jie, Yaoyuan Yan, Jiacheng Zhou, and Zhiyou Heng. Ev-wm: Event-verified world models for long-horizon robotic manipulation. *arXiv e-prints*, pages arXiv–2606, 2026.
- [36] Chenyang Ma, Guangyu Yang, Kai Lu, Shitong Xu, Bill Byrne, Niki Trigoni, and Andrew Markham. Cyclevla: Proactive self-correcting vision-language-action models via subtask backtracking and minimum bayes risk decoding. *arXiv preprint arXiv:2601.02295*, 2026.
- [37] Maximilian Du, Alexander Khazatsky, Tobias Gerstenberg, and Chelsea Finn. To err is robotic: Rapid value-based trial-and-error during deployment. *arXiv preprint arXiv:2406.15917*, 2024.
- [38] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- [39] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [40] Qualcomm ai hub apps. <https://github.com/quic/ai-hub-apps/tree/main>, 2025. Accessed: 2025-11-12.
- [41] Wei Zhao, Pengxiang Ding, Zhang Min, Zhefei Gong, Shuanghao Bai, Han Zhao, and Donglin Wang. Vlas: Vision-language-action model with speech instructions for customized robot manipulation. In *International conference on learning representations*, volume 2025, pages 51676–51693, 2025.
- [42] Yide Shentu, Philipp Wu, Aravind Rajeswaran, and Pieter Abbeel. From llms to actions: Latent codes as bridges in hierarchical robot control. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8539–8546. IEEE, 2024.
- [43] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
- [44] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. In *International Conference on Learning Representations*, volume 2024, pages 26703–26721, 2024.
- [45] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, volume 2024, pages 33431–33452, 2024.
- [46] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, volume 2025, pages 92033–92052, 2025.
- [47] Moritz Reuss, Jyothish Pari, Pulkit Agrawal, and Rudolf Lioutikov. Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning. In *International Conference on Learning Representations*, volume 2025, pages 17247–17275, 2025.
- [48] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- [49] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations*, volume 2026, pages 18567–18602, 2026.
- [50] Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdiñç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-action flow policies. *arXiv preprint arXiv:2509.04996*, 2025.
- [51] Yuqi Wang, Xinghang Li, Wenxuan Wang, Junbo Zhang, Yingyan Li, Yuntao Chen, Xinlong Wang, and Zhaoxiang Zhang. Unified vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 80929–80944, 2026.
- [52] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, et al. Vlada: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646, 2026.
- [53] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [54] Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue Wu, Sitong Mao, Liua Pei, Tian Nian, Shunbo Zhou, Xiaokang Yang, Jiangmiao Pang, et al. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025.
- [55] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [56] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Zhijun Li, Donglin Wang, Lujia Wang, et al. Pd-vla: Accelerating vision-language-action model integrated with action chunking via parallel decoding. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13162–13169. IEEE, 2025.
- [57] Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yujie Yuan, Xinyue Cai, Guowei Huang, Xingyue Quan, Hang Xu, et al. 4d-vla: Spatiotemporal vision-language-action pretraining with cross-scene calibration. *Advances in Neural Information Processing Systems*, 38:33914–33937, 2026.
- [58] Zhide Zhong, Haodong Yan, Junfeng Li, Xiangchen Liu, Xin Gong, Wenxuan Song, Jiayi Chen, and Haoqiang Li. Flowvla: Thinking in motion with a visual chain of thought. *arXiv e-prints*, pages arXiv–2508, 2025.
- [59] Yifan Yang, Zhixiang Duan, Tianshi Xie, Fuyu Cao, Pinxi Shen, Peili Song, Chenyang Zhao, Piaopiao Jin, Guokang Sun, Shaoqing Xu, et al. Fpc-vla: A vision-language-action framework with a supervisor for failure prediction and correction. *Expert Systems with Applications*, page 131742, 2026.
- [60] Jason Lee, Jiafei Duan, Haoqiang Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.
- [61] Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *Advances in Neural Information Processing Systems*, 38:82782–82802, 2026.
- [62] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [63] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, volume 2025, pages 54277–54296, 2025.

- [64] Chia-Yu Hung, Qi Sun, Pengfei Hong, Amir Zadeh, Chuan Li, U Tan, Navonil Majumder, Soujanya Poria, et al. Nora: A small open-sourced generalist vision language action model for embodied tasks. *arXiv preprint arXiv:2504.19854*, 2025.
- [65] Chongkai Gao, Zixuan Liu, Zhenghao Chi, Junshan Huang, Xin Fei, Yiwen Hou, Yuxuan Zhang, Yudi Lin, Zhirui Fang, and Lin Shao. Vla-os: Structuring and dissecting planning representations and paradigms in vision-language-action models. *Advances in Neural Information Processing Systems*, 38:136705–136736, 2026.
- [66] Jinliang Zheng, Jianxiong Li, Dongxiu Liu, Yanan Zheng, Zhihao Wang, Zhonghong Ou, Yu Liu, Jingjing Liu, Ya-Qin Zhang, and Xianyu Zhan. Universal actions for enhanced embodied foundation models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22508–22519. IEEE, 2025.
- [67] Qingwen Bu, Hongyang Li, Li Chen, Jisong Cai, Jia Zeng, Heming Cui, Maoqing Yao, and Yu Qiao. Towards synergistic, generalized, and efficient dual-system for robotic manipulation. *arXiv preprint arXiv:2410.08001*, 2024.
- [68] Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, et al. Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation. *arXiv preprint arXiv:2505.03912*, 2025.
- [69] Wenxuan Song, Ziyang Zhou, Han Zhao, Jiayi Chen, Pengxiang Ding, Haodong Yan, Yuxin Huang, Feilong Tang, Donglin Wang, and Haoang Li. Reconvla: Reconstructive vision-language-action model as effective robot perceiver. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 18549–18557, 2026.
- [70] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- [71] Xinghang Li, Peiyan Li, Long Qian, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, Xinlong Wang, Di Guo, et al. What matters in building vision-language-action models for generalist robots. *Nature Machine Intelligence*, 8(2):158–172, 2026.
- [72] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.

A APPENDIX

A.1 Dual-View Latent World Model Details

We provide additional architectural and training details for the dual-view latent world model introduced in Sec. 4.2. The model maintains separate latent representations for the agent-view and wrist-view streams while sharing action-conditioned dynamics across them.

A.1.1 Dual-View Architecture.

Visual preprocessing and encoding. At each time step t , the model receives an agent-view observation o_t^a , a wrist-view observation o_t^w , and robot action $\mathbf{a}_t$. Both images are resized to a common spatial resolution, scaled to $[0, 1]$, and normalized using ImageNet statistics. No random cropping or additional image augmentation is applied.

Each camera stream is processed by an independently parameterized visual encoder and projector:

$$\mathbf{z}_t^v = P_v(E_v(o_t^v)), \quad v \in \{a, w\}, \quad (39)$$

where $\mathbf{z}_t^v$ denotes the view-specific latent representation. The agent and wrist branches do not share visual-encoder or projector parameters.

Dual-view fusion. The two latent representations are independently normalized and augmented with learnable view embeddings:

$$\bar{\mathbf{z}}_t^a = \text{LN}_a(\mathbf{z}_t^a) + \mathbf{e}_a, \quad (40)$$

$$\bar{\mathbf{z}}_t^w = \text{LN}_w(\mathbf{z}_t^w) + \mathbf{e}_w. \quad (41)$$

The normalized representations are concatenated and projected into a shared dynamics space:

$$\mathbf{f}_t = F_{\text{fuse}}([\bar{\mathbf{z}}_t^a; \bar{\mathbf{z}}_t^w]), \quad (42)$$

where

$$F_{\text{fuse}}(\mathbf{x}) = W_{\text{fuse},2} \text{GELU}(\text{LN}(W_{\text{fuse},1}\mathbf{x})). \quad (43)$$

This design preserves view-specific visual information while allowing both camera streams to jointly influence the shared physical dynamics.

Action normalization and encoding. Robot actions are normalized using statistics computed from the world-model training set and stored with its checkpoint:

$$\tilde{\mathbf{a}}_t = \frac{\mathbf{a}_t - \mu_A^{\text{WM}}}{\max(\sigma_A^{\text{WM}}, \epsilon_A)}, \quad (44)$$

where the maximum is applied element-wise. The normalized action is mapped to the latent conditioning space through the action encoder:

$$\mathbf{c}_t = E_A(\tilde{\mathbf{a}}_t). \quad (45)$$

Shared action-conditioned dynamics. Given a latent history of length L , the shared dynamics predictor computes

$$\mathbf{h}_{t+1} = D(\mathbf{f}_{t-L+1:t}, \mathbf{c}_{t-L+1:t}), \quad (46)$$

where D is a causal Transformer. Actions are injected into each Transformer block using AdaLN-Zero. For hidden representation $\mathbf{x}$ and conditioning vector $\mathbf{c}$, the attention update is

$$\mathbf{x}' = \mathbf{x} + \mathbf{g}_{\text{attn}}(\mathbf{c}) \odot \text{Attn}(\text{LN}(\mathbf{x}) \odot [1 + \mathbf{s}_{\text{attn}}(\mathbf{c})] + \mathbf{b}_{\text{attn}}(\mathbf{c})), \quad (47)$$

followed by

$$\mathbf{x}'' = \mathbf{x}' + \mathbf{g}_{\text{mlp}}(\mathbf{c}) \odot \text{MLP}(\text{LN}(\mathbf{x}') \odot [1 + \mathbf{s}_{\text{mlp}}(\mathbf{c})] + \mathbf{b}_{\text{mlp}}(\mathbf{c})). \quad (48)$$

The final modulation layers are zero-initialized, allowing action-conditioned modulation to be introduced gradually during optimization.

View-specific residual prediction. The shared dynamics representation is decoded by independent residual heads for the two camera streams:

$$\widehat{\mathbf{z}}_{t+1}^v = \mathbf{z}_t^v + H_v(\mathbf{h}_{t+1}), \quad v \in \{a, w\}. \quad (49)$$

The output layer of each prediction head is zero-initialized. Consequently, the model initially satisfies

$$\widehat{\mathbf{z}}_{t+1}^v \approx \mathbf{z}_t^v, \quad (50)$$

and learns action-conditioned latent residuals from a copy-current-state initialization.

A.1.2 Generic Dual-View Pretraining. Generic pretraining jointly learns the visual representations, cross-view fusion, and shared action-conditioned dynamics. Training examples are constructed using short sliding windows. For each valid position in a window, the model performs one-step latent prediction.

One-step latent prediction. For each view $v \in \{a, w\}$, the prediction objective is

$$\mathcal{L}_{\text{MSE}}^v = \frac{1}{BT_p D_z} \sum_{b=1}^B \sum_{j=1}^{T_p} \left\| \widehat{\mathbf{z}}_{b,j}^v - \mathbf{z}_{b,j}^v \right\|_2^2, \quad (51)$$

where B denotes the batch size, T_p the number of prediction positions in each training window, and D_z the latent dimension.

The prediction targets are produced by the same trainable visual encoders and are not detached. No momentum or target encoder is used. Thus, gradients from the prediction loss jointly optimize the visual representation and latent dynamics.

SIGReg representation regularization. We additionally regularize each view-specific latent space using SIGReg. For a latent representation $\mathbf{z}_b$ and a randomly sampled unit direction $\mathbf{u}_p$, we define the one-dimensional projection

$$r_{b,p} = \mathbf{u}_p^\top \mathbf{z}_b. \quad (52)$$

For frequency s_k , the empirical characteristic function is

$$C_{p,k} = \frac{1}{B} \sum_{b=1}^B \cos(s_k r_{b,p}), \quad (53)$$

$$S_{p,k} = \frac{1}{B} \sum_{b=1}^B \sin(s_k r_{b,p}). \quad (54)$$

The characteristic function of a standard Gaussian is

$$\phi(s_k) = \exp\left(-\frac{s_k^2}{2}\right). \quad (55)$$

The SIGReg objective for view v is

$$\mathcal{L}_{\text{SIGReg}}^v = \frac{B}{T_p N_{\text{dir}}} \sum_{\tau=1}^{T_p} \sum_{p=1}^{N_{\text{dir}}} \sum_{k=0}^{N_{\text{freq}}-1} w_k \left[(C_{\tau,p,k} - \phi(s_k))^2 + S_{\tau,p,k}^2 \right], \quad (56)$$

where N_{dir} is the number of sampled projection directions, N_{freq} is the number of frequency points, and w_k denotes the corresponding trapezoidal integration weight.

Pretraining objective. The complete dual-view pretraining objective is

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{MSE}}^a + \lambda_w^{\text{pre}} \mathcal{L}_{\text{MSE}}^w + \lambda_{\text{reg}} \left(\mathcal{L}_{\text{SIGReg}}^a + \mathcal{L}_{\text{SIGReg}}^w \right), \quad (57)$$

corresponding to Eq. 6 in the main text.

Latent fusion, shared dynamics, and joint backpropagation from the two prediction heads couple the camera streams under the prediction and SIGReg objectives in Eq. 57.

A.1.3 Task-Specific Multi-Step Adaptation. After generic pretraining, the dual-view latent world model is adapted to multi-step dynamics of the target manipulation tasks. During adaptation, the visual encoders and projectors are frozen, while the fusion module, action-conditioning modules, view normalization and embeddings, shared dynamics predictor, and view-specific prediction heads remain trainable.

Task-specific conditioning augments the robot action with proprioceptive and task information:

$$\mathbf{c}_{t+h} = E_A(\widetilde{\mathbf{a}}_{t+h}) + E_p(\widetilde{\mathbf{p}}_t) + E_g(g), \quad (58)$$

where $\widetilde{\mathbf{p}}_t = \mathcal{N}_p(\mathbf{p}_t)$ applies the proprioceptive preprocessing used during task-specific adaptation, E_p denotes the proprioceptive adapter, and $E_g(g)$ denotes the task embedding. The proprioceptive condition is taken from the beginning of the rollout and shared across the prediction horizon.

Recursive latent rollout. Let the encoded ground-truth future trajectory for view v be

$$\mathbf{Z}^v = [\mathbf{z}_{t+1}^v, \dots, \mathbf{z}_{t+H}^v], \quad (59)$$

and let the recursively predicted trajectory be

$$\widehat{\mathbf{Z}}^v = [\widehat{\mathbf{z}}_{t+1}^v, \dots, \widehat{\mathbf{z}}_{t+H}^v]. \quad (60)$$

The first future state is predicted from the encoded observation history. Subsequent predictions recursively consume previously predicted latent states.

Recursive feature alignment. We align the recursively predicted and encoded future trajectories using both feature-value and directional supervision:

$$\mathcal{L}_{\text{feature}}^v = \text{SmoothL1} \left(\widehat{\mathbf{Z}}^v, \mathbf{Z}^v \right) + \lambda_{\text{cos}}^{\text{wm}} \left[1 - \text{CosSim} \left(\widehat{\mathbf{Z}}^v, \mathbf{Z}^v \right) \right]. \quad (61)$$

Smooth L1 is applied element-wise and averaged over the rollout.

Latent displacement alignment. In addition to matching the future latent states directly, we align their displacements relative to the initial state:

$$\Delta \widehat{\mathbf{z}}_h^v = \widehat{\mathbf{z}}_{t+h}^v - \mathbf{z}_t^v, \quad (62)$$

$$\Delta \mathbf{z}_h^v = \mathbf{z}_{t+h}^v - \mathbf{z}_t^v. \quad (63)$$

We evaluate the displacement objective over future steps with non-negligible target motion:

$$\mathcal{V}^v = \{h \mid \|\Delta \mathbf{z}_h^v\|_2 > \epsilon_\Delta\}. \quad (64)$$

The directional component is

$$\mathcal{L}_{\text{dir}}^v = \frac{1}{|\mathcal{V}^v|} \sum_{h \in \mathcal{V}^v} \left[1 - \frac{\left(\Delta \widehat{\mathbf{z}}_h^v \right)^\top \Delta \mathbf{z}_h^v}{\|\Delta \widehat{\mathbf{z}}_h^v\|_2 \|\Delta \mathbf{z}_h^v\|_2} \right], \quad (65)$$

while the magnitude component is

$$\mathcal{L}_{\text{mag}}^v = \frac{1}{|\mathcal{V}^v|} \sum_{h \in \mathcal{V}^v} \left| \log \frac{\max \left(\|\Delta \widehat{\mathbf{z}}_h^v\|_2, \epsilon \right)}{\max \left(\|\Delta \mathbf{z}_h^v\|_2, \epsilon \right)} \right|. \quad (66)$$

The resulting displacement loss is

$$\mathcal{L}_\Delta^v = \mathcal{L}_{\text{dir}}^v + \mathcal{L}_{\text{mag}}^v. \quad (67)$$

This objective constrains both the direction and magnitude of the latent trajectory relative to the initial observation.

Teacher-forced temporal supervision. Recursive rollout exposes the model to its own prediction errors. We therefore complement recursive prediction with teacher-forced one-step supervision. At each future step, the predictor receives the encoded ground-truth latent history rather than previously predicted latents:

$$\widehat{\mathbf{z}}_{t+h}^v = \mathcal{D}_v \left(\mathbf{z}_{t+h-L:t+h-1}^{a,w}, \mathbf{c}_{t+h-L:t+h-1} \right), \quad (68)$$

where $\mathcal{D}_v$ compactly denotes the shared fusion and dynamics modules followed by the view-specific residual prediction head. Collecting these predictions gives

$$\widetilde{\mathbf{Z}}^v = [\widehat{\mathbf{z}}_{t+1}^v, \dots, \widehat{\mathbf{z}}_{t+H}^v]. \quad (69)$$

The teacher-forced temporal objective is

$$\mathcal{L}_{\text{temporal}}^v = \text{SmoothL1} \left(\widetilde{\mathbf{Z}}^v, \mathbf{Z}^v \right) + \lambda_{\text{tf-cos}} \left[1 - \text{CosSim} \left(\widetilde{\mathbf{Z}}^v, \mathbf{Z}^v \right) \right]. \quad (70)$$

Task-adaptation objective. The loss for each view is

$$\mathcal{L}_v = \mathcal{L}_{\text{feature}}^v + \lambda_\Delta^{\text{wm}} \mathcal{L}_\Delta^v + \lambda_{\text{temp}} \mathcal{L}_{\text{temporal}}^v, \quad (71)$$

and the two views are jointly optimized using

$$\mathcal{L}_{\text{task}} = \mathcal{L}_a + \lambda_w^{\text{task}} \mathcal{L}_w, \quad (72)$$

corresponding to Eq. 7 in the main text.

A.1.4 Recursive Counterfactual Rollout. For counterfactual evaluation, each candidate action chunk $\mathbf{A}_t^k$ is converted to physical control units, truncated to $\mathbf{A}_{t,1:H}^k$, and normalized using the statistics stored with the world-model checkpoint. Starting from the synchronized real latent history and the fixed rollout context $(\bar{\mathbf{p}}_t, g)$, the dual-view latent world model predicts the future dual-view latent state

$$\widehat{\mathbf{z}}_{t+h}^k = \left(\widehat{\mathbf{z}}_{t+h}^{a,k}, \widehat{\mathbf{z}}_{t+h}^{w,k} \right), \quad h = 1, \dots, H, \quad (73)$$

forming the imagined trajectory

$$\widehat{\mathbf{z}}_{t+1:t+H}^k = \left[\widehat{\mathbf{z}}_{t+1}^k, \dots, \widehat{\mathbf{z}}_{t+H}^k \right]. \quad (74)$$

Only the initial history is obtained from real observations. After the first imagined step, predicted agent- and wrist-view latents are recursively fed back into the dynamics model together with the corresponding candidate actions. The resulting imagined trajectory therefore captures the multi-step latent consequences of each candidate before execution.

A.1.5 Implementation Details.

Architecture. Both visual streams use independently initialized ViT [53] encoders with no pretrained visual weights. Images are resized to 128×128 , and the CLS token is used as the visual representation. The latent dimension is $D_z = 192$.

Each visual projector uses

$$192 \rightarrow 768 \rightarrow 192, \quad (75)$$

with batch normalization and GELU. The dual-view fusion module maps the concatenated representation using

$$384 \rightarrow 768 \rightarrow 192. \quad (76)$$

The shared dynamics predictor contains six causal Transformer layers, six attention heads, hidden dimension 192, MLP dimension 768, and dropout 0.1. A learnable temporal positional embedding is used, and the history length is $L = 3$. Action conditioning is injected into each Transformer block through AdaLN-Zero.

Robot actions are seven-dimensional. The action encoder first applies a $7 \rightarrow 10$ pointwise convolution and then a SiLU MLP with dimensions

$$10 \rightarrow 768 \rightarrow 192. \quad (77)$$

Each view-specific residual prediction head uses

$$192 \rightarrow 768 \rightarrow 192 \quad (78)$$

with batch normalization and GELU. The final layer is zero-initialized.

Generic pretraining. Each one-step prediction position is constructed from a contiguous real history of $L = 3$ latent observations and its next-state target. History slots are never filled by replicating the current latent. SIGReg samples $N_{\text{dir}} = 256$ random projection directions and uses $N_{\text{freq}} = 17$ frequency points

$$s_k = \frac{3k}{N_{\text{freq}} - 1}, \quad k = 0, \dots, N_{\text{freq}} - 1. \quad (79)$$

The SIGReg integration weights use trapezoidal quadrature with the Gaussian weighting term $\exp(-s_k^2/2)$.

The pretraining objective uses

$$\lambda_w^{\text{pre}} = 0.5, \quad \lambda_{\text{reg}} = 0.09. \quad (80)$$

We train using AdamW with batch size 64, learning rate 5×10^{-5} , weight decay 10^{-3} , and gradient clipping at 1.0 for 100k optimization steps.

Task-specific adaptation. Task-specific adaptation uses a prediction horizon of $H = 5$. The two visual encoders and projectors are frozen, while the fusion module, dynamics predictor, action encoder, view normalization, view embeddings, residual prediction heads, proprioceptive adapter, and task embedding remain trainable.

Smooth L1 uses the standard unit transition point,

$$\ell_{\text{SL1}}(e) = \begin{cases} \frac{1}{2}e^2, & |e| < 1, \\ |e| - \frac{1}{2}, & \text{otherwise.} \end{cases} \quad (81)$$

The displacement mask uses

$$\epsilon_{\Delta} = 10^{-4}, \quad (82)$$

and numerical stabilization in the magnitude ratio uses

$$\epsilon = 10^{-6}. \quad (83)$$

The adaptation loss coefficients are

$$\lambda_{\text{cos}}^{\text{wm}} = 0.5, \lambda_{\text{tf-cos}} = 0.5, \lambda_{\Delta}^{\text{wm}} = 0.2, \lambda_{\text{temp}} = 0.5, \lambda_w^{\text{task}} = 0.5. \quad (84)$$

Adaptation uses AdamW with batch size 128. The dynamics modules use learning rate 10^{-5} , while the task and proprioceptive adapters use learning rate 3×10^{-4} . We use weight decay 10^{-4} , gradient clipping at 1.0, and train for 80 epochs.

Checkpoint selection. We select the task-adapted checkpoint using a validation score that combines recursive end-point accuracy, teacher-forced prediction accuracy, and latent-displacement direction:

$$\begin{aligned} S = & \text{MSE}_{\text{end}}^a + 0.5 \text{MSE}_{\text{end}}^w \\ & + 0.25 (\text{MSE}_{\text{TF}}^a + 0.5 \text{MSE}_{\text{TF}}^w) \\ & + 0.01 [(1 - \cos_{\Delta}^a) + 0.5 (1 - \cos_{\Delta}^w)]. \end{aligned} \quad (85)$$

The checkpoint with the lowest validation score is retained.

A.2 Student Policy Distillation Details

A.2.1 Representation Distillation Objectives. We provide the detailed formulation of the representation distillation objectives used in Stage 1. At time step t , let $\mathbf{F}_t^T \in \mathbb{R}^{N \times d_T}$ and $\mathbf{F}_t^S \in \mathbb{R}^{N \times d_S}$ denote the Teacher and Student intermediate features, respectively, where N is the number of feature tokens. When their feature dimensions differ, we project the Student features into the Teacher feature space:

$$\tilde{\mathbf{F}}_t^S = P(\mathbf{F}_t^S) \in \mathbb{R}^{N \times d_T}, \quad (86)$$

where P is the Student feature projection layer. The Teacher branch is frozen and treated as a stop-gradient target throughout representation distillation.

Feature value alignment. We first directly align the projected Student features with the corresponding Teacher features:

$$\mathcal{L}_{\text{feat}} = \frac{1}{Nd_T} \left\| \tilde{\mathbf{F}}_t^S - \text{sg}(\mathbf{F}_t^T) \right\|_F^2, \quad (87)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operation. This term constrains the Student to reproduce the Teacher feature values in the aligned representation space.

Feature directional alignment. Direct feature matching can be sensitive to differences in feature magnitude. We therefore additionally align the direction of each feature token:

$$\mathcal{L}_{\text{cos}} = \frac{1}{N} \sum_{n=1}^N \left(1 - \frac{(\tilde{\mathbf{f}}_{t,n}^S)^\top \text{sg}(\mathbf{f}_{t,n}^T)}{\|\tilde{\mathbf{f}}_{t,n}^S\|_2 \|\text{sg}(\mathbf{f}_{t,n}^T)\|_2 + \epsilon} \right), \quad (88)$$

where $\mathbf{f}_{t,n}$ denotes the feature vector of token n . This term complements feature-value matching by reducing sensitivity to representation scale.

Token-relation preservation. Matching individual tokens does not explicitly preserve the relational structure among tokens. To retain this structure, we first normalize each feature token along the feature dimension and construct pairwise similarity matrices

$$\mathbf{R}_t^T = \bar{\mathbf{F}}_t^T (\bar{\mathbf{F}}_t^T)^\top, \quad (89)$$

$$\mathbf{R}_t^S = \bar{\mathbf{F}}_t^S (\bar{\mathbf{F}}_t^S)^\top, \quad (90)$$

where each row of $\bar{\mathbf{F}}$ is ℓ_2 -normalized. We then minimize

$$\mathcal{L}_{\text{rel}} = \frac{1}{N^2} \left\| \mathbf{R}_t^S - \text{sg}(\mathbf{R}_t^T) \right\|_F^2. \quad (91)$$

This objective encourages the Student to preserve the pairwise structure encoded across Teacher feature tokens, rather than matching each token independently.

Action-head projection alignment. Because the distilled representation is ultimately consumed by the action-generation module, we additionally align the Teacher and Student representations after the projection used by the downstream

action head. Let $\Pi(\cdot)$ denote this action-head projection and define

$$\mathbf{U}_t^T = \Pi(\mathbf{F}_t^T), \quad (92)$$

$$\mathbf{U}_t^S = \Pi(\tilde{\mathbf{F}}_t^S). \quad (93)$$

We minimize

$$\mathcal{L}_{\text{proj}} = \frac{1}{|\mathbf{U}_t|} \left\| \mathbf{U}_t^S - \text{sg}(\mathbf{U}_t^T) \right\|_F^2, \quad (94)$$

where $|\mathbf{U}_t|$ denotes the number of scalar elements in the projected representation. This term explicitly aligns the representations in the feature space observed by the downstream action generator.

Temporal dynamics alignment. Frame-wise alignment alone does not constrain how the representation changes as the trajectory evolves. We therefore compare feature differences across observations separated by a temporal offset δ :

$$\Delta_\delta \mathbf{F}_t^T = \text{sg}(\mathbf{F}_{t+\delta}^T - \mathbf{F}_t^T), \quad (95)$$

$$\Delta_\delta \tilde{\mathbf{F}}_t^S = \tilde{\mathbf{F}}_{t+\delta}^S - \tilde{\mathbf{F}}_t^S. \quad (96)$$

The temporal distillation loss is

$$\mathcal{L}_\Delta^{\text{rep}} = \frac{1}{Nd_T} \left\| \Delta_\delta \tilde{\mathbf{F}}_t^S - \Delta_\delta \mathbf{F}_t^T \right\|_F^2. \quad (97)$$

This objective encourages the Student to preserve the temporal evolution of the Teacher representation in addition to its instantaneous features.

Combined representation objective. Combining the above terms yields the representation objective

$$\mathcal{L}_{\text{rep}} = \lambda_{\text{feat}} \mathcal{L}_{\text{feat}} + \lambda_{\text{cos}} \mathcal{L}_{\text{cos}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}} + \lambda_{\text{proj}} \mathcal{L}_{\text{proj}} + \lambda_\Delta^{\text{rep}} \mathcal{L}_\Delta^{\text{rep}}, \quad (98)$$

which corresponds to Eq. 9 in the main text.

A.2.2 Action-Aware Representation Refinement. Representation alignment alone does not ensure that the distilled features retain all information required by the downstream action generator. After the initial representation-only phase, we therefore augment Stage 1 with action-aware supervision while keeping the action-generation module fixed.

Let $\mathbf{c}_\pi^T$ and $\mathbf{c}_\pi^S$ denote the Teacher and aligned Student conditioning representations supplied to the frozen flow-based action predictor, respectively. For the same noisy action $\mathbf{x}_\tau$, proprioceptive state $\mathbf{p}$, and flow timestep τ , let

$$\mathbf{v}_\tau^T = v(\mathbf{x}_\tau, \tau, \mathbf{c}_\pi^T, \mathbf{p}), \quad (99)$$

$$\mathbf{v}_\tau^S = v(\mathbf{x}_\tau, \tau, \mathbf{c}_\pi^S, \mathbf{p}) \quad (100)$$

denote the Teacher- and Student-conditioned flow predictions produced by the same frozen action predictor.

Velocity consistency. We first require the Student representation to induce a velocity field consistent with that obtained under Teacher conditioning:

$$\mathcal{L}_{\text{vel-cons}} = \frac{1}{H_a D_a} \left\| \mathbf{v}_\tau^S - \text{sg}(\mathbf{v}_\tau^T) \right\|_F^2, \quad (101)$$

where H_a and D_a denote the action horizon and action dimension. Because both predictions share the same action predictor, noisy action, and flow timestep, this loss isolates the effect of replacing the Teacher representation with the Student representation.

Dataset-flow supervision. Velocity consistency alone can reproduce Teacher-conditioned behavior without explicitly anchoring the prediction to the demonstrated action. Let $\mathbf{A}^D$ denote the dataset action chunk. We sample

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (102)$$

and construct

$$\mathbf{x}_\tau = (1 - \tau) \mathbf{A}^D + \tau \boldsymbol{\epsilon}, \quad (103)$$

with target flow velocity

$$\mathbf{v}^* = \boldsymbol{\epsilon} - \mathbf{A}^D. \quad (104)$$

We then define

$$\mathcal{L}_{\text{flow-GT}} = \frac{1}{H_a D_a} \left\| \mathbf{v}_\tau^S - \mathbf{v}^* \right\|_F^2. \quad (105)$$

This term anchors the Student-conditioned action prediction to the dataset trajectory while preserving the representation-focused role of Stage 1.

Action-aware objective. The action-aware refinement objective is

$$\mathcal{L}_{\text{aware}} = \lambda_{\text{vel}} \mathcal{L}_{\text{vel-cons}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow-GT}}, \quad (106)$$

and the later phase of Stage 1 optimizes

$$\mathcal{L}_{S1} = \mathcal{L}_{\text{rep}} + \lambda_{\text{aware}} \mathcal{L}_{\text{aware}}, \quad (107)$$

as defined in Eq. 10. During this phase, the Teacher branch and action-generation module remain frozen; gradients from the action-aware objectives are propagated through the Student representation pathway. Thus, these losses refine the control relevance of the Student representation rather than directly distilling the Student action head.

A.2.3 Action Distillation Objectives. Stage 2 directly distills the Teacher action-generation behavior into the Student action head. Throughout this stage, both the Teacher and Student action heads are conditioned on the same fixed Teacher visual-language tokens, eliminating representation mismatch from the action-distillation objective.

Let

$$\mathbf{A}^D = [\mathbf{a}_1^D, \dots, \mathbf{a}_{H_a}^D] \in \mathbb{R}^{H_a \times D_a} \quad (108)$$

denote the dataset action chunk, and let $\mathbf{A}^T$ denote the corresponding action generated by the Teacher. We denote the fixed Teacher conditioning tokens by $\mathbf{c}_\pi^T$ and the proprioceptive state by $\mathbf{p}$.

Given Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and flow timestep τ , we construct

$$\mathbf{x}_\tau = (1 - \tau) \mathbf{A}^D + \tau \boldsymbol{\epsilon}, \quad (109)$$

with the corresponding dataset flow target

$$\mathbf{v}^* = \boldsymbol{\epsilon} - \mathbf{A}^D. \quad (110)$$

The Student and Teacher velocity predictions are

$$\mathbf{v}_\tau^S = v_S(\mathbf{x}_\tau, \tau, \mathbf{c}_\pi^T, \mathbf{p}), \quad (111)$$

$$\mathbf{v}_\tau^T = v_T(\mathbf{x}_\tau, \tau, \mathbf{c}_\pi^T, \mathbf{p}). \quad (112)$$

Critical-step weighting. Not all steps in an action chunk are equally important for downstream execution. We assign each action step h a weight

$$w_h = 1 + \lambda_{\text{div}} m_h^{\text{first-div}} + \lambda_{\text{trans}} m_h^{\text{transition}} + \lambda_{\text{grasp}} m_h^{\text{grasp}}, \quad (113)$$

where $m_h^{\text{first-div}}$ marks the first action step at which the Student prediction diverges from the dataset action, $m_h^{\text{transition}}$ marks a gripper-state transition, and m_h^{grasp} marks a grasp event. The first-divergence step can occur anywhere within the action chunk.

For a per-step loss ℓ_h , we use the normalized weighted reduction

$$\langle \ell \rangle_w = \frac{\sum_{h=1}^{H_a} w_h \ell_h}{\sum_{h=1}^{H_a} w_h}. \quad (114)$$

Velocity distillation. We first transfer the Teacher velocity field by matching the Student and Teacher predictions under the same noisy action and flow timestep:

$$\mathcal{L}_{\text{vel-KD}} = \left\langle \frac{1}{D_a} \left\| \mathbf{v}_{\tau,h}^S - \text{sg}(\mathbf{v}_{\tau,h}^T) \right\|_2^2 \right\rangle_w. \quad (115)$$

This objective transfers the local flow field learned by the Teacher.

Dataset-flow supervision. We additionally supervise the Student velocity using the flow target induced by the dataset action:

$$\mathcal{L}_{\text{flow-GT}} = \left\langle \frac{1}{D_a} \left\| \mathbf{v}_{\tau,h}^S - \mathbf{v}_h^* \right\|_2^2 \right\rangle_w. \quad (116)$$

Together, velocity distillation and dataset-flow supervision transfer the Teacher generation dynamics while retaining direct grounding in demonstrated behavior.

Flow-endpoint distillation. The predicted velocities also imply clean-action endpoints. For the Student and Teacher, respectively, we compute

$$\hat{\mathbf{A}}_{\text{end}}^S = \mathbf{x}_\tau - \tau \mathbf{v}_\tau^S, \quad (117)$$

$$\hat{\mathbf{A}}_{\text{end}}^T = \mathbf{x}_\tau - \tau \mathbf{v}_\tau^T. \quad (118)$$

We then align the corresponding endpoints:

$$\mathcal{L}_{\text{end-KD}} = \left\langle \text{SmoothL1} \left(\hat{\mathbf{a}}_{\text{end},h}^S, \text{sg}(\hat{\mathbf{a}}_{\text{end},h}^T) \right) \right\rangle_w. \quad (119)$$

This term complements velocity-field matching by directly constraining the clean action implied by the predicted flow.

Dataset action supervision. In addition to flow-level supervision, we directly constrain the Student-generated action chunk using the dataset trajectory. Let

$$\hat{\mathbf{A}}^S = [\hat{\mathbf{a}}_1^S, \dots, \hat{\mathbf{a}}_{H_a}^S] \quad (120)$$

denote the action chunk produced by the Student flow model. We define

$$\mathcal{L}_{\text{action-GT}} = \left\langle \text{SmoothL1} \left(\hat{\mathbf{a}}_h^S, \mathbf{a}_h^D \right) \right\rangle_w. \quad (121)$$

This term directly anchors the final Student action prediction to the demonstrated trajectory.

Gripper supervision. Gripper transitions are sparse but behaviorally important and can be underrepresented by losses averaged over the full action vector. We therefore apply an additional loss to the gripper component:

$$\mathcal{L}_{\text{grip}} = \left\langle \ell_{\text{grip}} \left(\hat{u}_{\text{grip},h}^S, u_{\text{grip},h}^D \right) \right\rangle_w, \quad (122)$$

where $\hat{u}_{\text{grip},h}^S$ and $u_{\text{grip},h}^D$ denote the Student and dataset gripper outputs, respectively, and ℓ_{grip} is the loss associated with the gripper parameterization used by the policy.

Combined action objective. The complete Stage 2 objective is

$$\mathcal{L}_{\text{act}} = \lambda_{\text{vel}} \mathcal{L}_{\text{vel-KD}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow-GT}} + \lambda_{\text{end}} \mathcal{L}_{\text{end-KD}} + \lambda_{\text{action}} \mathcal{L}_{\text{action-GT}} + \lambda_{\text{grip}} \mathcal{L}_{\text{grip}}, \quad (123)$$

which corresponds to Eq. 11 in the main text.

Throughout Stage 2, the Teacher visual-language representation and Teacher action module remain fixed, while gradients are used to optimize the Student action head. The Student predicts the complete action chunk directly; the Teacher action is used only as a distillation target and is never added to the Student prediction as a residual correction.

A.2.4 On-Policy Joint Refinement.

Dual-view latent history. The dual-view latent world model operates on synchronized agent-view and wrist-view observations. For each time step τ , we define

$$\mathbf{z}_\tau^a = P_a(E_a(o_\tau^a)), \quad \mathbf{z}_\tau^w = P_w(E_w(o_\tau^w)), \quad \mathbf{Z}_\tau = (\mathbf{z}_\tau^a, \mathbf{z}_\tau^w), \quad (124)$$

where E_a, E_w and P_a, P_w are the frozen visual encoders and projectors of the dual-view latent world model.

The dual-view latent world model conditions future prediction on the most recent L dual-view latent observations:

$$\mathbf{Z}_{t-L+1:t} = \{\mathbf{Z}_\tau\}_{\tau=t-L+1}^t. \quad (125)$$

In the current configuration, $L = 3$. The same latent history is used for both the Student-conditioned and Teacher-conditioned world-model branches.

Differentiable Student and Teacher actions. Stage 3 constructs Student and Teacher corrective actions from the same flow-matching intermediate state. Let $\bar{\mathbf{A}}_t^{\text{roll}}$ denote the normalized action chunk associated with the on-policy sample. We draw

$$\tau \sim \text{Beta}(1.5, 1), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (126)$$

and construct

$$\mathbf{x}_\tau = \tau \epsilon + (1 - \tau) \bar{\mathbf{A}}_t^{\text{roll}}. \quad (127)$$

Conditioned on the Student and Teacher representations, respectively, the two action heads predict

$$\mathbf{v}_t^S = v_\theta(\mathbf{x}_t, \tau, \mathbf{F}_t^S, \mathbf{p}_t), \quad (128)$$

$$\mathbf{v}_t^T = v_T(\mathbf{x}_t, \tau, \mathbf{F}_t^T, \mathbf{p}_t), \quad (129)$$

where $\mathbf{p}_t$ denotes the robot proprioceptive state.

The corresponding differentiable endpoint action chunks are

$$\hat{\mathbf{A}}_t^S = \mathbf{x}_t - \tau \mathbf{v}_t^S, \quad (130)$$

$$\bar{\mathbf{A}}_t^T = \mathbf{x}_t - \tau \mathbf{v}_t^T. \quad (131)$$

Both chunks contain ten action steps with seven action dimensions. The dual-view latent world model evaluates only the first H actions, with $H = 5$ in the current configuration.

Action normalization for the dual-view latent world model. The policy and dual-view latent world model need not use the same action statistics. We therefore first convert the endpoint actions back to physical action units using the policy normalization, and then normalize them using the statistics stored with the dual-view latent world model checkpoint. Let

$$\hat{\mathbf{A}}_t^S = \mathcal{N}_\pi^{-1}(\hat{\mathbf{A}}_t^S), \quad (132)$$

$$\bar{\mathbf{A}}_t^T = \mathcal{N}_\pi^{-1}(\bar{\mathbf{A}}_t^T). \quad (133)$$

Both branches then apply the single world-model normalization $\mathcal{N}_A$ defined in Eq. 44, using $\boldsymbol{\mu}_A^{\text{WM}}$, $\boldsymbol{\sigma}_A^{\text{WM}}$, and ϵ_A loaded from the dual-view latent world model checkpoint.

Future prediction. Using the same latent context for both branches, the dual-view latent world model predicts future agent-view and wrist-view representations:

$$\{\hat{\mathbf{z}}_{t+h}^{a,S}, \hat{\mathbf{z}}_{t+h}^{w,S}\}_{h=1}^H = \mathcal{W}_\omega(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\hat{\mathbf{A}}_{t,1:H}^S), \tilde{\mathbf{p}}_t, g), \quad (134)$$

and

$$\{\hat{\mathbf{z}}_{t+h}^{a,T}, \hat{\mathbf{z}}_{t+h}^{w,T}\}_{h=1}^H = \text{sg}\left[\mathcal{W}_\omega(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\bar{\mathbf{A}}_{t,1:H}^T), \tilde{\mathbf{p}}_t, g)\right]. \quad (135)$$

The world-model parameters ω are frozen in both branches. The Teacher-conditioned future is detached, whereas the Student-conditioned branch remains differentiable with respect to the Student action:

$$\mathcal{L}_{\text{WM}} \rightarrow \hat{\mathbf{Z}}^S \rightarrow \hat{\mathbf{A}}^S \rightarrow \theta. \quad (136)$$

Latent future consistency. For a predicted latent $\hat{\mathbf{z}}$ and Teacher-conditioned target $\mathbf{z}^*$, we first define

$$\ell_{\text{feat}}(\hat{\mathbf{z}}, \mathbf{z}^*) = \text{SmoothL1}(\hat{\mathbf{z}}, \mathbf{z}^*) + 0.5 \left(1 - \frac{\hat{\mathbf{z}}^\top \mathbf{z}^*}{\|\hat{\mathbf{z}}\|_2 \|\mathbf{z}^*\|_2 + \epsilon}\right). \quad (137)$$

We additionally compare the displacement of each predicted future from the current latent. For view $v \in \{a, w\}$, define

$$\Delta \hat{\mathbf{z}} = \hat{\mathbf{z}} - \mathbf{z}_t^v, \quad \Delta \mathbf{z}^* = \mathbf{z}^* - \mathbf{z}_t^v. \quad (138)$$

The displacement discrepancy is

$$\ell_\Delta(\hat{\mathbf{z}}, \mathbf{z}^*) = 1 - \cos(\Delta \hat{\mathbf{z}}, \Delta \mathbf{z}^*) + \left| \log \frac{\|\Delta \hat{\mathbf{z}}\|_2 + \epsilon}{\|\Delta \mathbf{z}^*\|_2 + \epsilon} \right|. \quad (139)$$

We combine the two terms as

$$\ell_{\text{latent}}(\hat{\mathbf{z}}, \mathbf{z}^*) = \ell_{\text{feat}}(\hat{\mathbf{z}}, \mathbf{z}^*) + 0.2 \ell_\Delta(\hat{\mathbf{z}}, \mathbf{z}^*). \quad (140)$$

This formulation follows the feature and displacement objectives used to train the dual-view latent world model, while avoiding teacher-forced temporal supervision that would require ground-truth future observations.

The resulting world-model objective is

$$\mathcal{L}_{\text{WM}} = \frac{1}{\sum_{h=1}^H \gamma^{h-1}} \sum_{h=1}^H \gamma^{h-1} \left[\ell_{\text{latent}}(\hat{\mathbf{z}}_{t+h}^{a,S}, \hat{\mathbf{z}}_{t+h}^{a,T}) + \lambda_w^{\text{S3}} \ell_{\text{latent}}(\hat{\mathbf{z}}_{t+h}^{w,S}, \hat{\mathbf{z}}_{t+h}^{w,T}) \right]. \quad (141)$$

The current dual-view latent world model configuration uses $H = 5$ and $\lambda_w^{\text{S3}} = 0.5$. The discount γ controls the relative importance of near- and longer-horizon predictions and is treated as a Stage 3 hyperparameter.

On-policy representation objective. For an on-policy observation at time t and a valid subsequent observation t' , $\mathbf{F}^S$ denotes the Student feature after the Stage 1 alignment projection and $\mathbf{F}^T$ denotes the corresponding frozen Teacher feature. Let

$$\bar{\mathbf{F}}_t^S = \text{LN}(\mathbf{F}_t^S), \quad \bar{\mathbf{F}}_t^T = \text{LN}(\mathbf{F}_t^T). \quad (142)$$

The feature-value objective is

$$\mathcal{L}_{\text{feat}}^{\text{on}} = \frac{1}{2} \left[\text{MSE}(\bar{\mathbf{F}}_t^S, \text{sg}(\bar{\mathbf{F}}_{t'}^T)) + \text{MSE}(\bar{\mathbf{F}}_{t'}^S, \text{sg}(\bar{\mathbf{F}}_t^T)) \right]. \quad (143)$$

The directional objective is

$$\mathcal{L}_{\text{cos}}^{\text{on}} = \frac{1}{2} \left[1 - \cos(\mathbf{F}_t^S, \text{sg}(\mathbf{F}_{t'}^T)) + 1 - \cos(\mathbf{F}_{t'}^S, \text{sg}(\mathbf{F}_t^T)) \right], \quad (144)$$

and the temporal objective is

$$\mathcal{L}_\Delta^{\text{on}} = \text{MSE}(\bar{\mathbf{F}}_{t'}^S - \bar{\mathbf{F}}_t^S, \text{sg}(\bar{\mathbf{F}}_{t'}^T - \bar{\mathbf{F}}_t^T)). \quad (145)$$

The temporal term is evaluated only for pairs marked valid by the temporal-pair mask. The implemented on-policy representation objective is

$$\mathcal{L}_{\text{rep}}^{\text{on}} = 0.5 \mathcal{L}_{\text{feat}}^{\text{on}} + 0.1 \mathcal{L}_{\text{cos}}^{\text{on}} + 0.25 \mathcal{L}_\Delta^{\text{on}}. \quad (146)$$

On-policy corrective action objective. Stage 3 also distills Teacher corrective actions at Student-induced states. For trajectory b and action step j , we define

$$w_{b,j} = w_b^{\text{fail}} \left(1 + 2 \mathbb{I}[j = j_b^{\text{first}}] + 2 \mathbb{I}[u_{\text{grip},b,j}^S \neq u_{\text{grip},b,j}^T] \right), \quad (147)$$

where j_b^{first} is the first action step at which the Student diverges from the Teacher correction. A trajectory-level weight w_b^{fail} gives additional emphasis to failed Student rollouts. In the current implementation, $w_b^{\text{fail}} = 2$ for failed trajectories and 1 otherwise.

For a per-step squared error $\ell_{b,j}$, we define the weighted mean

$$\text{WMSE}(\ell; \mathbf{w}) = \frac{\sum_{b,j} w_{b,j} \ell_{b,j}}{\sum_{b,j} w_{b,j}}. \quad (148)$$

The velocity and endpoint objectives are then

$$\mathcal{L}_{\text{vel}}^{\text{on}} = \text{WMSE} \left(\left\| \mathbf{v}_t^S - \text{sg} \left(\mathbf{v}_t^T \right) \right\|_2^2; \mathbf{w} \right), \quad (149)$$

$$\mathcal{L}_{\text{end}}^{\text{on}} = \text{WMSE} \left(\left\| \hat{\mathbf{A}}_t^S - \text{sg} \left(\bar{\mathbf{A}}_t^T \right) \right\|_2^2; \mathbf{w} \right). \quad (150)$$

An additional gripper objective $\mathcal{L}_{\text{grip}}^{\text{on}}$ emphasizes disagreement in the gripper component. The implemented corrective action objective is

$$\mathcal{L}_{\text{act}}^{\text{on}} = \mathcal{L}_{\text{vel}}^{\text{on}} + 0.25 \mathcal{L}_{\text{end}}^{\text{on}} + 0.1 \mathcal{L}_{\text{grip}}^{\text{on}}. \quad (151)$$

Combined Stage 3 objective. The complete on-policy refinement objective is

$$\mathcal{L}_{\text{S3}} = \mathcal{L}_{\text{rep}}^{\text{on}} + \lambda_{\text{act}} \mathcal{L}_{\text{act}}^{\text{on}} + \lambda_{\text{WM}} \mathcal{L}_{\text{WM}}, \quad (152)$$

which corresponds to Eq. 15 in the main text. The representation and action terms are already defined by the on-policy refinement procedure, whereas λ_{WM} controls the contribution of the additional future-consistency regularizer.

Throughout Stage 3, the Teacher policy, dual-view latent world model encoders, and world-model parameters remain frozen. Gradients from $\mathcal{L}_{\text{rep}}^{\text{on}}$ and $\mathcal{L}_{\text{act}}^{\text{on}}$ directly update the Student, while $\mathcal{L}_{\text{WM}}$ additionally propagates through the frozen Student-conditioned world-model computation to the Student action generator.

A.3 Task-Semantic State Modeling Details

This section details the trajectory construction, supervision, and frozen-model boundary for the task-semantic estimator in Sec. 4.4.

Observed-plus-predicted trajectory. For every anchor time t , the estimator uses the same synchronized real dual-view history as the world model. For view $v \in \{a, w\}$ and candidate k , we construct

$$\mathbf{z}_{t-L+1:t+H}^{v,k} = \left[\mathbf{z}_{t-L+1}^v, \dots, \mathbf{z}_t^v, \hat{\mathbf{z}}_{t+1}^{v,k}, \dots, \hat{\mathbf{z}}_{t+H}^{v,k} \right], \quad (153)$$

where the first L latents are encoded from real observations and the remaining H latents are predicted from $\mathbf{A}_{t,1:H}^k$ by the frozen dual-view latent world model. The current configuration uses $L = 3$ and $H = 5$. Agent-view and wrist-view frames are synchronized, and nominal/candidate comparisons reuse the identical observed context.

We form first-order changes over the complete sequence,

$$\Delta \mathbf{z}_{\tau}^{v,k} = \mathbf{z}_{\tau}^{v,k} - \mathbf{z}_{\tau-1}^{v,k}, \quad \tau = t - L + 2, \dots, t + H, \quad (154)$$

and pass the two view sequences, their temporal changes, the current normalized proprioceptive state, the normalized candidate prefix, and the task embedding to the semantic encoder:

$$\begin{aligned} \mathbf{x}_{\text{sem}}^k &= \text{concat}(\mathbf{z}_{t-L+1:t+H}^{a,k}, \mathbf{z}_{t-L+1:t+H}^{w,k}, \\ \Delta \mathbf{z}_{t-L+2:t+H}^{a,k}, \Delta \mathbf{z}_{t-L+2:t+H}^{w,k}, \tilde{\mathbf{p}}_t, \tilde{\mathbf{A}}_{t,1:H}^k, E_g(g)). \end{aligned} \quad (155)$$

Each estimator input therefore preserves the real multi-frame context and all predicted intermediate states.

Task-grounded supervision. For task g , each training item contains task-grounded predicate labels $\mathbf{y} \in \{0, 1\}^{M_g}$ and, where available, continuous geometric margins $\mathbf{m}$. The predicate definitions specify the physical condition and temporal semantics represented by each label. Simulation data derives these targets from environment states and task predicates; real-world data uses the corresponding physical measurements or annotations. Both sources use the same estimator interface and loss names.

Let $\ell = f_{\text{pred}}(\mathbf{r}_{\text{sem}})$ denote predicate logits and $\hat{\mathbf{m}} = f_{\text{margin}}(\mathbf{r}_{\text{sem}})$ the predicted margins, where $\mathbf{r}_{\text{sem}} = f_{\text{sem}}(\mathbf{x}_{\text{sem}})$. The training objective is

$$\mathcal{L}_{\text{pred}} = \frac{1}{M_g} \sum_{j=1}^{M_g} w_j \text{BCEWithLogits}(\ell_j, y_j), \quad (156)$$

$$\mathcal{L}_{\text{margin}} = \text{SmoothL1}(\hat{\mathbf{m}}, \mathbf{m}), \quad (157)$$

$$\mathcal{L}_{\text{sem}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{margin}} \mathcal{L}_{\text{margin}}, \quad (158)$$

matching Eqs. 21–23.

Frozen inference boundary. The dual-view latent world model and its visual encoders/projectors remain frozen throughout semantic-estimator training. Predicted trajectories are detached and may be cached, so gradients from $\mathcal{L}_{\text{sem}}$ update only f_{sem} and its predicate and margin heads. During gate training and deployment, the semantic estimator is also frozen and acts only as a feature extractor.

A.4 Action-Conditioned Continuation Value

We provide the detailed architecture, supervision, and training procedure for the action-conditioned continuation-value estimator introduced in Sec. 4.5.

Trajectory construction. At time t , let

$$\mathbf{Z}_t = (\mathbf{z}_t^a, \mathbf{z}_t^w) \quad (159)$$

denote the agent-view and wrist-view latent state. For candidate action chunk $\mathbf{A}_t$, the frozen dual-view latent world model predicts

$$\hat{\mathbf{Z}}_{t+1:t+H}^A = \mathcal{W}_{\omega}(\mathbf{Z}_{t-L+1:t}, \mathcal{N}_A(\mathbf{A}_{t,1:H}), \tilde{\mathbf{p}}_t, g). \quad (160)$$

The continuation-value model receives the combined observed and imagined trajectory

$$\mathbf{X}_t^{A,\text{imag}} = [\mathbf{Z}_{t-L+1:t}, \hat{\mathbf{Z}}_{t+1:t+H}^A]. \quad (161)$$

The current implementation uses $L = 3$ observed states and $H = 5$ predicted states, yielding eight temporal positions.

Value encoder. Each view-specific latent has dimension 192. At each temporal position τ , we first fuse the two views:

$$\mathbf{u}_{\tau} = f_{\text{fuse}}([\text{LN}(\mathbf{z}_{\tau}^a); \text{LN}(\mathbf{z}_{\tau}^w)]) \in \mathbb{R}^{256}, \quad (162)$$

where f_{fuse} is a two-layer projection from 384 to 256 dimensions. For imagined positions, $\mathbf{z}_{\tau}^a$ and $\mathbf{z}_{\tau}^w$ are replaced by the corresponding world-model predictions.

We add temporal and source embeddings to distinguish temporal position and whether a token is observed or predicted:

$$\mathbf{e}_\tau = \mathbf{u}_\tau + \mathbf{e}_\tau^{\text{time}} + \mathbf{e}_\tau^{\text{source}}. \quad (163)$$

Only observed proprioceptive history is used:

$$\mathbf{c}_p = f_p(\text{vec}[\mathbf{p}_{t-L+1:t}]) \in \mathbb{R}^{256}. \quad (164)$$

We inject this context into a learnable CLS token,

$$\mathbf{e}_{\text{CLS}} = \mathbf{e}_{\text{CLS}}^0 + \mathbf{c}_p, \quad (165)$$

and process

$$[\mathbf{e}_{\text{CLS}}, \mathbf{e}_{t-L+1}, \dots, \mathbf{e}_{t+H}] \quad (166)$$

with a Transformer encoder. The current implementation uses a 256-dimensional hidden representation, two Transformer layers, eight attention heads, a feed-forward dimension of 1024, and dropout of 0.1.

Let $\mathbf{h}_{\text{CLS}}^A$ denote the resulting CLS representation. Three heads produce

$$q_t^A = \mathbf{w}_Q^\top \mathbf{h}_{\text{CLS}}^A + b_Q, \quad (167)$$

$$\hat{\rho}_{t+H}^A = \sigma(\mathbf{w}_P^\top \mathbf{h}_{\text{CLS}}^A + b_P), \quad (168)$$

$$\mathbf{r}_t^A = f_{\text{repr}}(\mathbf{h}_{\text{CLS}}^A). \quad (169)$$

Before calibration, the continuation-success training probability is $Q_t^A = \sigma(q_t^A)$. We write $f_{\text{value}, \psi}$ for the joint encoder/head mapping and Q_ψ for its success-probability output. The progress prediction is auxiliary, whereas $\mathbf{r}_t^A$ is provided to the downstream gate.

Candidate-specific continuation labels. Each training sample contains an observed history, candidate action, world-model-predicted future, and the outcome obtained after executing the same H -step prefix used by deployment and then continuing with the frozen Student:

$$(\mathbf{Z}_{t-L+1:t}, \mathbf{p}_{t-L+1:t}, \mathbf{A}_t, \hat{\mathbf{Z}}_{t+1:t+H}^A, Y_t^A, \rho_{t+H}^A). \quad (170)$$

The success target is

$$Y_t^A = \mathbb{I}[\text{execute } \mathbf{A}_{t,1:H} \text{ and then follow } \pi_\theta \text{ succeeds}]. \quad (171)$$

When the same candidate is evaluated using N_{rep} continuation rollouts, we use the empirical success probability

$$\bar{Y}_t^A = \frac{1}{N_{\text{rep}}} \sum_{r=1}^{N_{\text{rep}}} Y_{t,r}^A. \quad (172)$$

This record format is shared across data sources: simulation branches obtain terminal success and progress from task state, whereas real-world branches use the corresponding physical measurements or annotations. Candidate-specific continuation labels are essential: propagating the nominal outcome to all candidates from the same state would prevent the model from learning action-dependent differences.

Monte-Carlo value pretraining. The primary success objective is

$$\mathcal{L}_{\text{MC}} = \text{BCEWithLogits}(q_t^A, \bar{Y}_t^A). \quad (173)$$

The auxiliary progress head predicts progress at the end of the prediction horizon:

$$\mathcal{L}_{\text{prog}} = \text{SmoothL1}(\hat{\rho}_{t+H}^A, \rho_{t+H}^{\text{real}}). \quad (174)$$

This progress signal provides denser supervision but never replaces the terminal success label.

For a successful and unsuccessful candidate sampled from the same anchor state, we additionally impose

$$\mathcal{L}_{\text{rank}} = \text{softplus}(- (q_t^{A+} - q_t^{A-})). \quad (175)$$

The first training phase minimizes

$$\mathcal{L}_{V1} = \mathcal{L}_{\text{MC}} + 0.2\mathcal{L}_{\text{prog}} + 0.1\mathcal{L}_{\text{rank}}. \quad (176)$$

Temporal-difference refinement. Monte-Carlo outcomes provide terminal supervision but do not explicitly enforce consistency between successive planning points. We therefore maintain an exponential-moving-average target model with parameters $\bar{\psi}$:

$$\bar{\psi} \leftarrow (1 - \tau_{\text{EMA}})\bar{\psi} + \tau_{\text{EMA}}\psi. \quad (177)$$

After executing candidate prefix $\mathbf{A}_{t,1:H}$, we construct the next value input from the realized state at $t+H$ and the Student’s nominal continuation:

$$\mathbf{X}_{t+H}^{\text{nom}} = [\mathbf{Z}_{t+H-L+1:t+H}^{\text{real}}, \hat{\mathbf{Z}}_{t+H+1:t+2H}^{\text{nom}}]. \quad (178)$$

The temporal-difference target is

$$y_t^{\text{TD}} = d_t \bar{Y}_t^A + (1 - d_t) \text{sg} \left[Q_{\bar{\psi}}(\mathbf{X}_{t+H}^{\text{nom}}, \mathbf{p}_{t+H-L+1:t+H}) \right], \quad (179)$$

where $d_t = 1$ when the trajectory terminates within the current continuation segment. Because the model estimates eventual success probability rather than discounted return, we do not discount the bootstrap target.

The corresponding loss is

$$\mathcal{L}_{\text{TD}} = \text{BCEWithLogits}(q_t^A, y_t^{\text{TD}}). \quad (180)$$

The second phase minimizes

$$\mathcal{L}_{V2} = 0.5\mathcal{L}_{\text{MC}} + 0.5\mathcal{L}_{\text{TD}} + 0.2\mathcal{L}_{\text{prog}} + 0.1\mathcal{L}_{\text{rank}}. \quad (181)$$

Retaining the Monte-Carlo objective limits accumulation of bootstrap error.

Imagined-real value alignment. Deployment evaluates candidates using world-model-predicted futures, whereas executed training branches additionally provide realized future trajectories. We use these paired trajectories to reduce the resulting imagined-to-real value shift.

For an executed candidate, we construct

$$\mathbf{X}_t^{A, \text{imag}} = [\mathbf{Z}_{t-L+1:t}, \hat{\mathbf{Z}}_{t+1:t+H}^A], \quad (182)$$

$$\mathbf{X}_t^{A, \text{real}} = [\mathbf{Z}_{t-L+1:t}, \mathbf{Z}_{t+1:t+H}^{\text{real}}]. \quad (183)$$

The online model processes the imagined trajectory, while the EMA target model processes the realized trajectory:

$$q_t^I, \mathbf{r}_t^I = f_{\text{value}, \psi}(\mathbf{X}_t^{A, \text{imag}}, \mathbf{p}_{t-L+1:t}), \quad (184)$$

$$q_t^R, \mathbf{r}_t^R = \text{sg} \left[f_{\text{value}, \psi}(\mathbf{X}_t^{A, \text{real}}, \mathbf{p}_{t-L+1:t}) \right]. \quad (185)$$

We align both the value logits and internal representations:

$$\mathcal{L}_{\text{align}} = \text{SmoothL1}(q_t^I, q_t^R) + 0.1(1 - \cos(\mathbf{r}_t^I, \mathbf{r}_t^R)). \quad (186)$$

Aligning logits rather than post-sigmoid probabilities preserves useful gradients in high-confidence regions.

The final training phase minimizes

$$\mathcal{L}_{V3} = 0.3\mathcal{L}_{\text{MC}} + 0.5\mathcal{L}_{\text{TD}} + 0.2\mathcal{L}_{\text{prog}} + 0.1\mathcal{L}_{\text{rank}} + 0.5\mathcal{L}_{\text{align}}. \quad (187)$$

Optimization details. The first phase trains the continuation-value estimator for 40,000 steps with learning rate 3×10^{-4} . The second phase initializes from the best first-phase checkpoint and trains for another 40,000 steps with learning rate 10^{-4} . The final alignment phase trains for 20,000 steps with learning rate 5×10^{-5} . We use AdamW, weight decay 10^{-4} , gradient clipping at 1.0, and $\tau_{\text{EMA}} = 0.005$. Sampling balances nominal and candidate branches and avoids allowing long trajectories to dominate optimization.

The dual-view latent world model and its visual encoders remain frozen throughout all three phases. World-model predictions used by the continuation-value estimator are detached and may be precomputed. Thus, continuation-value training never propagates gradients through the world model, in contrast to the future-consistency objective used during Student refinement.

Data splitting and calibration. Baseline trajectories, candidate branches, and continuation rollouts originating from the same anchor are always assigned to the same data split to prevent leakage. Model selection is performed on a held-out validation split using success discrimination, calibration, and candidate-ranking metrics. After selecting the final checkpoint, we calibrate the continuation probability using temperature scaling on the validation set:

$$Q_{\text{cal}} = \sigma\left(\frac{q}{T_{\text{cal}}}\right). \quad (188)$$

where T_{cal} is fitted on the validation split. Held-out evaluation trajectories are not used for training, checkpoint selection, calibration, or threshold selection. All deployment references to Q_t^k below denote this calibrated probability.

Deployment. At each replanning step, the Student first produces and evaluates only the nominal action. Alternative candidates are sampled only if the calibrated nominal risk crosses τ_{risk} and the current decision-phase budget permits a counterfactual batch. The cached nominal trajectory is reused, and the alternative trajectories are evaluated in one batch. For every evaluated branch,

$$Q_k = \sigma\left(\frac{1}{T_{\text{cal}}} q_{\psi}(\mathbf{Z}_{t-L+1:t}, \hat{\mathbf{Z}}_{t+1:t+H}^k, \mathbf{p}_{t-L+1:t})\right). \quad (189)$$

Relative to the nominal candidate A_t^0 , we compute

$$\Delta Q_k = Q_k - Q_0. \quad (190)$$

The continuation probabilities, relative gains, progress estimates, and value representations are passed to the downstream gate together with the semantic and risk signals. The gate replaces the nominal action only when the predicted benefit is sufficient under the corresponding safety constraints.

A.5 Risk-Aware Counterfactual Intervention

We provide additional details on risk-triggered candidate generation, deterministic action filtering, rescue-harm supervision, gate training, and intervention calibration.

Risk-triggered candidate generation. The deployed trigger directly uses the action-conditioned continuation value of the nominal Student proposal:

$$R_t = 1 - Q_t^0, \quad (191)$$

with

$$Q_t^0 = \sigma\left(\frac{1}{T_{\text{cal}}} q_{\psi}(\mathbf{Z}_{t-2:t}, \hat{\mathbf{Z}}_{t+1:t+5}^0, \mathbf{p}_{t-2:t})\right). \quad (192)$$

Candidate generation is activated when

$$R_t \geq \tau_{\text{risk}}. \quad (193)$$

The trigger threshold is selected on validation trajectories to balance failure coverage against counterfactual-computation overhead. In the current deployment, candidate generation is further constrained to predefined decision phases, with at most one counterfactual batch generated per phase. Thus, repeated low-value states within the same phase do not repeatedly invoke expensive candidate evaluation.

Candidate resampling. When the trigger fires, the nominal proposal A_t^0 is retained and $K - 1$ additional action chunks are independently sampled from the same frozen Student policy, giving a total candidate budget of K . We use $K = 8$ by default, i.e., one nominal proposal and seven additional alternatives:

$$A_t^k \sim \pi_{\theta}(\cdot | s_t), \quad k = 1, \dots, K - 1. \quad (194)$$

All candidates use the same current observation and policy parameters and differ only in flow-sampling randomness. We de-normalize policy outputs to physical control units before applying $\mathcal{N}_A$; A_t^k below denotes this physical-unit chunk.

Each candidate is evaluated using a five-step future predicted by the frozen dual-view latent world model:

$$\hat{\mathbf{Z}}_{t+1:t+5}^k = \mathcal{W}_{\omega}(\mathbf{Z}_{t-2:t}, \mathcal{N}_A(A_{t,1:5}^k), \tilde{\mathbf{p}}_t, g). \quad (195)$$

The complete observed-plus-predicted trajectory is passed to the continuation-value and task-semantic estimators.

Deterministic action filtering. Before invoking the learned gate, each candidate must contain only finite action values and remain sufficiently close to the nominal Student action. For candidate $\mathbf{A}_t^k$, we compute its mean and maximum pose deviations, d_{mean}^k and d_{max}^k , and the number of action steps whose gripper command differs from the nominal proposal, n_{grip}^k .

The current deployment requires

$$d_{\text{mean}}^k \leq 0.25, \quad d_{\text{max}}^k \leq 0.60, \quad n_{\text{grip}}^k \leq 4. \quad (196)$$

Candidates violating any constraint are removed before learned selection. If all alternatives are rejected, the nominal proposal is executed.

Counterfactual branch labels. Gate supervision is obtained from paired counterfactual continuations. For repeat r , let

$$Y_{0,r} \in \{0, 1\}, \quad Y_{k,r} \in \{0, 1\} \quad (197)$$

denote terminal success after executing the nominal and candidate H -step prefixes, respectively, followed by Student continuation.

We define

$$Y_{\text{rescue},r}^k = (1 - Y_{0,r})Y_{k,r}, \quad (198)$$

$$Y_{\text{harm},r}^k = Y_{0,r}(1 - Y_{k,r}), \quad (199)$$

corresponding to rescue and harm events. When multiple continuation repeats are available, we aggregate them into soft labels:

$$y_{\text{rescue}}^k = \frac{1}{N_{\text{rep}}} \sum_{r=1}^{N_{\text{rep}}} Y_{\text{rescue},r}^k, \quad y_{\text{harm}}^k = \frac{1}{N_{\text{rep}}} \sum_{r=1}^{N_{\text{rep}}} Y_{\text{harm},r}^k, \quad (200)$$

$$y_0 = \frac{1}{N_{\text{rep}}} \sum_{r=1}^{N_{\text{rep}}} Y_{0,r}, \quad y_k = \frac{1}{N_{\text{rep}}} \sum_{r=1}^{N_{\text{rep}}} Y_{k,r}. \quad (201)$$

The expected terminal gain is

$$y_{\text{gain}}^k = y_k - y_0 = y_{\text{rescue}}^k - y_{\text{harm}}^k. \quad (202)$$

Gate feature construction. Each gate input represents a candidate jointly with its nominal counterpart. The continuation-value features include

$$[Q_k, Q_0, Q_k - Q_0, \hat{\rho}_k, \hat{\rho}_0, \hat{\rho}_k - \hat{\rho}_0], \quad (203)$$

together with

$$[\mathbf{r}_k^V, \mathbf{r}_0^V, \mathbf{r}_k^V - \mathbf{r}_0^V, |\mathbf{r}_k^V - \mathbf{r}_0^V|], \quad (204)$$

where $\mathbf{r}^V$ denotes the representation produced by the continuation-value encoder.

The task-semantic features similarly contain

$$[\mathbf{r}_{\text{sem}}^k, \mathbf{r}_{\text{sem}}^0, \mathbf{r}_{\text{sem}}^k - \mathbf{r}_{\text{sem}}^0, |\mathbf{r}_{\text{sem}}^k - \mathbf{r}_{\text{sem}}^0|], \quad (205)$$

together with candidate and nominal predicate probabilities, geometric margins, and their differences.

We additionally include action-deviation statistics, query position, and decision-phase information. With the current representation dimensions, the resulting gate feature vector has dimension 2098.

Gate architecture. The feature vector is normalized and processed by a two-layer MLP with 256 hidden units per layer, GELU activations, and dropout. A shared gate representation $\mathbf{h}$ feeds five output heads:

$$\hat{p}_{\text{rescue}} = \sigma(f_{\text{rescue}}(\mathbf{h})), \quad \hat{p}_{\text{harm}} = \sigma(f_{\text{harm}}(\mathbf{h})), \quad (206)$$

$$\widehat{\Delta Y} = \tanh(f_{\text{gain}}(\mathbf{h})), \quad \hat{p}_{\text{cand}} = \sigma(f_{\text{cand}}(\mathbf{h})), \quad (207)$$

$$\hat{p}_{\text{nom}} = \sigma(f_{\text{nom}}(\mathbf{h})). \quad (208)$$

The first three outputs drive the rescue-harm gate's deployment selection. The candidate- and nominal-success heads are auxiliary training heads for the shared representation and introduce no additional deployment thresholds.

Gate learning objective. The gate is optimized using

$$\mathcal{L}_{\text{gate}} = \mathcal{L}_{\text{rescue}} + \lambda_{\text{harm}} \mathcal{L}_{\text{harm}} + \lambda_{\text{succ}} (\mathcal{L}_{\text{cand}} + \mathcal{L}_{\text{nom}}) + \mathcal{L}_{\text{gain}} + \lambda_{\Delta}^{\text{gate}} \mathcal{L}_{\Delta}^{\text{gate}}, \quad (209)$$

where

$$\mathcal{L}_{\text{rescue}} = \text{BCE}(\hat{p}_{\text{rescue}}, y_{\text{rescue}}), \quad (210)$$

$$\mathcal{L}_{\text{harm}} = \text{BCE}(\hat{p}_{\text{harm}}, y_{\text{harm}}), \quad (211)$$

$$\mathcal{L}_{\text{cand}} = \text{BCE}(\hat{p}_{\text{cand}}, y_k), \quad (212)$$

$$\mathcal{L}_{\text{nom}} = \text{BCE}(\hat{p}_{\text{nom}}, y_0), \quad (213)$$

$$\mathcal{L}_{\text{gain}} = \text{SmoothL1}(\widehat{\Delta Y}, y_{\text{gain}}), \quad (214)$$

$$\mathcal{L}_{\Delta}^{\text{gate}} = \text{SmoothL1}(\hat{p}_{\text{cand}} - \hat{p}_{\text{nom}}, y_{\text{gain}}). \quad (215)$$

The current implementation uses

$$\lambda_{\text{harm}} = 3, \quad \lambda_{\text{succ}} = 0.25, \quad \lambda_{\Delta}^{\text{gate}} = 0.5, \quad (216)$$

assigning substantially greater cost to harm-prediction errors than to rescue errors.

The gate is trained for 40,000 optimization steps with batch size 256 using AdamW, learning rate 3×10^{-4} , weight decay 10^{-3} , dropout 0.30, and gradient clipping at 1.0. The Student policy, dual-view latent world model, continuation-value estimator, and task-semantic estimator remain frozen throughout gate training.

Cross-fitted gate training. To prevent leakage between related counterfactual branches, gate training is grouped by anchor initialization or physical reset. We use five folds indexed by this group identifier. For fold f , groups assigned to f are reserved for testing, the subsequent fold is used for validation, and the remaining folds are used for training.

Consequently, every evaluated initialization is processed by a gate that has not observed any nominal, candidate, or continuation branch associated with that initialization during training.

Risk-threshold calibration. The risk threshold τ_{risk} is selected using validation trajectories. Candidate thresholds are scanned over empirical risk quantiles. Calibration favors thresholds that achieve high recall on failed executions while limiting the fraction of decision phases that invoke counterfactual candidate generation.

Thus, the trigger is optimized not simply to identify every low-value state, but to concentrate computation on likely failures while bounding intervention overhead.

Gate-threshold calibration. We jointly calibrate

$$(\tau_{\text{rescue}}, \tau_{\text{harm}}, \tau_{\text{gain}}) \quad (217)$$

on held-out candidate branches. A candidate is eligible only if

$$\hat{p}_{\text{rescue}}^k \geq \tau_{\text{rescue}}, \quad \hat{p}_{\text{harm}}^k \leq \tau_{\text{harm}}, \quad \widehat{\Delta Y}^k \geq \tau_{\text{gain}}. \quad (218)$$

Among eligible candidates, the current implementation ranks

$$U_k = \hat{p}_{\text{rescue}}^k - 3\hat{p}_{\text{harm}}^k + 0.25\widehat{\Delta Y}^k. \quad (219)$$

This instantiates $\alpha_{\text{harm}} = 3$ and $\alpha_{\text{gain}} = 0.25$ in Eq. 37; these ranking weights are distinct from the gate-loss coefficients in Eq. 209. Threshold selection prioritizes configurations with a larger number of observed rescue events than harm events. Ties are resolved by favoring more rescues, fewer harms, fewer interventions, and larger expected gain.

Online intervention logic. Let $\mathcal{S}_t$ be the alternatives that pass the deterministic action filter, and let

$$\mathcal{E}_t = \left\{ k \in \mathcal{S}_t \mid \hat{p}_{\text{rescue}}^k \geq \tau_{\text{rescue}}, \hat{p}_{\text{harm}}^k \leq \tau_{\text{harm}}, \widehat{\Delta Y}^k \geq \tau_{\text{gain}} \right\} \quad (220)$$

be the learned-eligible set. If $\mathcal{E}_t$ is nonempty, we select

$$k^* = \arg \max_{k \in \mathcal{E}_t} U_k. \quad (221)$$

Let b_t^{phase} indicate that the per-phase counterfactual budget is available. The intervention condition is

$$I_t^{\text{int}} = \mathbb{I}[1 - Q_t^0 \geq \tau_{\text{risk}}] \mathbb{I}[b_t^{\text{phase}} = 1] \mathbb{I}[\mathcal{E}_t \neq \emptyset]. \quad (222)$$

The task-semantic outputs and continuation-value difference ΔQ_k are learned gate features rather than independent hard thresholds. If $I_t^{\text{int}} = 1$, $A_t^{k^*}$ replaces the nominal proposal; otherwise, A_t^0 is retained.

Receding-horizon execution. At each replanning point, the Student first produces a nominal proposal whose continuation value determines whether counterfactual reasoning is triggered. When triggered, the Student resamples alternative candidates, the dual-view latent world model predicts their futures, and the task-semantic and continuation-value estimators provide features to the rescue-harm gate. The controller executes the first $H = 5$ actions of the candidate accepted by the gate, or of the nominal proposal when no candidate is accepted, and then reacquires agent-view, wrist-view, and proprioceptive observations before replanning.

In shadow mode, the complete intervention pipeline is evaluated and proposed interventions are recorded while the nominal Student action is always executed. In active mode, candidates accepted by the learned gate are allowed to replace the nominal proposal.

Table 10: LIBERO results with audited full inference-time parameter counts. Success rates are reported in percent.

Method	Full Parameters	Spatial	Object	Goal	Long	Avg.
Large models ($P \geq 4$ B)						
OpenVLA-OFT [22]	≈ 7.71 B	97.6	98.4	97.9	94.5	97.1
MemoryVLA [49]	≈ 8.38 B	98.4	98.4	96.4	93.4	96.7
DD-VLA [54]	≈ 7.54 B	97.2	98.6	97.4	92.0	96.3
UnifiedVLA [51]	≈ 8.80 B	95.4	98.8	93.6	94.0	95.5
UniVLA [55]	≈ 7.55 B	96.5	96.8	95.6	92.0	95.2
PD-VLA [56]	≈ 7.21 B	95.5	96.7	94.9	91.7	94.7
4D-VLA [57]	> 4.15 B	88.9	95.2	90.9	79.1	88.5
FlowVLA [58]	≈ 8.80 B	93.2	95.0	91.6	72.6	88.1
FPC-VLA [59]	> 15.83 B	86.2	87.0	92.0	82.2	86.9
MolmoAct [60]	≈ 8.12 B	87.0	95.4	87.6	77.2	86.8
ThinkAct [61]	≥ 8.72 B	88.3	91.4	87.1	70.9	84.4
WorldVLA [62]	≈ 7.09 B	87.6	96.2	83.4	60.0	81.8
CoT-VLA [32]	≈ 8.12 B	87.5	91.6	87.6	69.0	83.9
SpatialVLA [48]	≈ 4.04 B	88.2	89.9	78.6	55.5	78.1
OpenVLA [1]	≈ 7.54 B	84.7	88.4	79.2	53.7	76.5
TraceVLA [63]	≈ 7.59 B	84.6	85.2	75.1	54.1	74.8
Small models ($1 \text{ B} \leq P < 4 \text{ B}$)						
VLA-Adapter-Pro [52]	≈ 1.47 B	99.6	99.6	98.2	96.4	98.5
VLA-Adapter [52]	≈ 1.36 B	97.8	99.2	97.2	95.0	97.3
$\pi_{0.5}$ [2]	≈ 3.62 B	98.8	98.2	98.0	92.4	96.9
π_0 [9]	≈ 3.50 B	96.8	98.8	95.8	85.2	94.2
GR00T N1 [10]	2.20 B	94.4	97.6	93.0	90.6	93.9
SmolVLA-2.25B [19]	2.25 B	93.0	94.0	91.0	77.0	88.8
NORA [64]	≈ 3.75 B	92.2	95.4	89.4	74.6	87.9
VLA-OS [65]	≈ 1.30 B	87.0	96.5	92.7	66.0	85.6
π_0 -FAST [14]	≈ 2.92 B	96.4	96.8	88.6	60.2	85.5
UniAct [66]	≈ 1.04 B	77.0	87.0	77.0	70.0	77.8
FLOWER [50]	≈ 1.00 B	97.1	96.7	95.6	93.5	95.7
Tiny models ($P < 1 \text{ B}$)						
SimVLA [4]	≈ 0.81 B	99.6	99.8	98.6	96.4	98.6
X-VLA [3]	≈ 0.88 B	98.2	98.6	97.8	97.6	98.1
Octo [17]	≈ 0.203 B	78.9	85.7	84.6	51.1	75.1
PicoVISTA (Ours)	≈ 0.078 B	98.4	99.2	94.6	93.2	96.4

B EVALUATION DETAILS

Extended benchmark comparisons. Tables 10, 11, and 12 complement the compact main-paper results with broader comparisons on LIBERO, CALVIN, and WidowX, respectively. The main paper retains representative baselines to preserve readability, whereas the appendix includes additional published methods using the corresponding evaluation protocol and benchmark-specific metrics. These extended comparisons provide additional context without changing the conclusions drawn from the main-paper results.

World-model cost. Table 13 reports world-model capacity, peak memory, and five-step recursive rollout latency on the target edge platform. The latency measurement covers the world-model rollout under the same input-history and action-horizon settings; it does not include the rest of the intervention pipeline.

The task-adapted dual-view latent world model uses 16.79M parameters, 99.55 MiB of peak memory, and 12.80 ms for a

Table 11: Comparison on the CALVIN ABC→D benchmark with audited full inference-time parameter counts. The five task columns report the percentage of evaluation sequences that complete at least the indicated number of consecutive tasks.

Method	Full Parameters	Tasks completed in a row $\uparrow$					Avg. len. $\uparrow$
		1	2	3	4	5	
Large models ($P \geq 4$ B)							
UniVLA [55]	≈ 7.554 B	95.5	85.8	75.4	66.9	56.5	3.80
OpenVLA [1]	≈ 7.54 B	91.3	77.8	62.0	52.1	43.5	3.27
OpenVLA-OFT [22]	≈ 7.71 B	96.3	89.1	82.4	75.8	66.5	4.10
VLAS [41]	> 7.063 B	87.2	64.2	40.9	28.1	19.6	2.40
LCB [42]	> 7.063 B	73.6	50.2	28.5	16.0	9.9	1.78
RoboDual [67]	≈ 7.580 B	94.4	82.7	72.1	62.4	54.4	3.66
OpenHelix [68]	≈ 7.175 B	97.1	91.4	82.8	72.6	64.1	4.08
ReconVLA [69]	≈ 8.209 B	95.6	87.6	76.9	69.3	64.1	3.95
Small models ($1\text{ B} \leq P < 4$ B)							
DeeR [43]	> 3.00 B	86.2	70.1	51.8	41.5	30.4	2.82
RoboFlamingo [44]	> 3.00 B	82.4	61.9	46.6	33.1	23.5	2.48
VPP [70]	> 1.50 B	95.7	91.2	86.3	81.0	75.0	4.29
SuSIE [45]	≈ 1.30 B	87.0	69.0	49.0	38.0	26.0	2.69
$\pi_{0.5}$ [2]	≈ 3.62 B	99.6	96.7	92.5	89.4	82.8	4.61
Tiny models ($P < 1$ B)							
Seer-Large [46]	≈ 0.566 B	96.3	91.6	86.1	80.3	74.0	4.28
MoDE [47]	≈ 0.740 B	96.2	88.9	81.1	71.8	63.5	4.01
Seer [46]	≈ 0.316 B	94.4	87.2	79.9	72.2	64.3	3.98
PicoVISTA (Ours)	≈ 0.078 B	96.2	91.1	83.4	76.5	62.3	4.10

Table 12: Comparison on SimplerEnv WidowX robot tasks with audited full inference-time parameter counts. Success rates are reported in percent.

Method	Full Parameters	Spoon	Carrot	Stack	Eggplant	Avg.
Large models ($P \geq 4$ B)						
OpenVLA [1]	≈ 7.54 B	0.0	0.0	0.0	4.1	1.0
OpenVLA-OFT [22]	≈ 7.71 B	12.5	4.2	8.3	37.5	15.6
DD-VLA [54]	≈ 7.54 B	29.2	29.2	20.8	70.8	37.5
SpatialVLA [48]	≈ 4.03 B	16.7	25.0	29.2	100.0	42.7
MemoryVLA [49]	≈ 8.38 B	75.0	75.0	37.5	100.0	71.9
ThinkAct [61]	≥ 8.72 B	58.3	37.5	8.7	70.8	43.8
FPC-VLA [59]	> 15.83 B	58.3	45.8	79.2	75.0	64.6
Small models ($1 \text{ B} \leq P < 4 \text{ B}$)						
RoboVLM [71]	> 3.00 B	29.2	25.0	12.5	58.3	31.3
π_0 [9]	≈ 3.50 B	29.1	0.0	16.7	62.5	27.1
π_0 -FAST [14]	≈ 2.92 B	29.1	21.9	10.8	66.6	32.1
GR00T N1 [10]	≈ 2.19 B	62.5	45.8	16.7	20.8	36.5
FLOWER [50]	≈ 1.00 B	71.0	13.0	8.0	88.0	45.0
Tiny models ($P < 1 \text{ B}$)						
RT-1-X [72]	≈ 0.292 B	0.0	4.2	0.0	0.0	1.1
Octo-Base [17]	≈ 0.203 B	12.5	8.3	0.0	43.1	16.0
Octo-Small [17]	≈ 0.14 B	47.2	9.7	4.2	56.9	29.5
X-VLA [3]	≈ 0.88 B	100.0	91.7	95.8	95.8	95.8
SimVLA [4]	≈ 0.81 B	100.0	91.7	91.7	100.0	95.8
PicoVISTA (Ours)	≈ 0.078 B	95.8	91.7	83.3	79.2	87.5

five-step rollout. Relative to DINO-WM, these point estimates reduce parameter count by 60.3%, peak memory by

Table 13: World-model resource consumption and five-step recursive rollout latency on the target edge platform. $\checkmark$ and $\times$ indicate that a component is enabled and disabled, respectively.

World model	Dual view	Target-task adapt.	Params	Peak memory (MiB)	WM $H=5$ latency (ms)
DINO-WM	$\times$	$\times$	42.25M	292.33	17.00
LeWM	$\times$	$\times$	10.25M	74.00	13.01
Dual-view latent world model w/o adaptation	$\checkmark$	$\times$	16.79M	99.55	13.13
Dual-view latent world model	$\checkmark$	$\checkmark$	16.79M	99.55	12.80

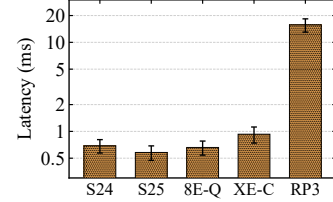


Figure 13: Batch-one ResNet-18 inference latency across the five deployment devices under a common measurement protocol. Lower is better.

65.9%, and rollout latency by 24.7%, while the corresponding pipeline improves average task success by 1.9 points. LeWM is smaller, but its 13.01 ms rollout latency is similar to that of the dual-view latent world model and its pipeline success is lower. The adapted and unadapted dual-view variants have identical capacity and similar measured latency, showing that the adaptation gain does not require additional deployment-time model capacity.

Device capability characterization. We benchmark all five devices using ResNet-18 as a standardized device-level workload. Figure 13 reports the batch-one inference latency measured under an identical deployment and profiling protocol. The comparison captures differences in neural-network execution capability across the mobile, reference-design, and low-cost edge platforms, providing a controlled hardware baseline for interpreting the cross-device latency results of PicoVISTA.